Mathematisch-Naturwissenschaftliche Fakultät

Lernbasierte Computer Vision

Masterarbeit

LLM-based Academic Writing with Fine-Grained Citations

Eberhard Karls Universität Tübingen
Mathematisch-Naturwissenschaftliche Fakultät
Wilhelm-Schickard-Institut für Informatik
Lernbasierte Computer Vision
Niklas Munkes, niklas.munkes@student.uni-tuebingen.de, 2025

Bearbeitungszeitraum: 14.04.-14.10.2025

Gutachter: Prof. Dr. Andreas Geiger, Universität Tübingen
Zweitgutachter: Prof. Dr. Carsten Eickhoff, Universität Tübingen
Betreuer: Haoyu He, Universität Tübingen

Selbstständigkeitserklärung

Hiermit versichere ich, dass ich die vorliegende Masterarbeit selbständig und nur mit den angegebenen Hilfsmitteln angefertigt habe und dass alle Stellen, die dem Wortlaut oder dem Sinne nach anderen Werken entnommen sind, durch Angaben von Quellen als Entlehnung kenntlich gemacht worden sind. Diese Masterarbeit wurde in gleicher oder ähnlicher Form in keinem anderen Studiengang als Prüfungsleistung vorgelegt.

Niklas Munkes (Matrikelnummer 4269436), October 9, 2025

Abstract

TODO

Contents

1. Introduction

Retrieval-Augmented Generation (RAG, [LPP+20]) models are designed to generate text that is grounded in real-world factual knowlege. They have been widely adopted for knowledge-intensive NLP tasks [SQK+25] such as long-form question answering [XWL+24], scientific question answering [LOS+23] and question synthesis [XLS+24], generation with sentence-level citations [ZBL+24, XWL+24] and academic text generation [AHS+24, WMN+25]. RAG models usually consist of a generator, a pre-trained sequence-to-sequence (seq2seq) model (e.g. a Transformer-based [VSP+17] decoder), and a retriever with access to a retrieval corpus, which may be implemented as a dense vector index (e.g. a HNSW [MY18] index) [LPP+20]. The RAG architecture is briefly introduced in section 3.1. Such models combine pre-trained paramentric memory with non-paramentric (retrieval-based) memory [LPP+20], which has a number of benefits: while neural language models are able to store a substantial amount of world knowledge in their parameters [RRS20], this knowledge can't easily be modified, expanded or directly be used to gain insight into their decision-making process [LLG+19]. By using an external retrieval corpus, the knowledge RAG models use to inform their generation can directly be inspected and interpreted [LPP+20]. Their TODO

citations build trust [DOW+24]

TODO

TODO

2. Related Work

TODO

2.1. LLM-based Generation

[STB25] TODO

2.2. RAG for Scientific Writing

Previous work on text generation with citations has recently transitioned from treating retrieval and generation as seperate processing stages (e.g. [SHC+24]) to alternating between the two (e.g. [XWL+24], [AHS+24]). *Attribute First, then Generate* [SHC+24] initially retrieves a set of relevant sub-sentence spans which are then grouped into coherent clusters which in turn are used to inform an iterative sentence-by-sentence generation step [SHC+24]. *ReClaim* [XWL+24], more specifically the *ReClaim with Interleaving Generation* method, is used to generate an output sequence $O = \{r_1, c_1, r_2, c_2, ..., r_n, c_n\}$ of alternating references and claims where claim c_i is the portion of the models response that was generated based on reference r_i. Claim and reference generation is performed by seperate LLMs specifically trained for their corresponding task [XWL+24]. In contrast, *OpenScholar* [AHS+24] first retrieves and reranks a set of papers P relevant to the given query. These papers inform a subsequent generation step that produces a generated response r_i [AHS+24]. This response is refined through iterative *self-feedback* where during each iteration of the feedback loop $F = f_1, f_2, ..., f_T$, the generator model generates sentences f_j that describe potential improvements to the response and may also prompt the retrieval pipeline to retrieve additional papers, should r_i require more information to generate an improved response r_{i+1} [AHS+24]. Depending on the size of P and the choice of T, OpenScholar still performs retrieval and generation largely seperately.

ScholarCopilot [WMN+25] is a RAG system developed for generating academic-style text with accurate citations. It is intended as a tool that assists the user with academic paper writing by providing iterative text generation with automated citation retrieval [WMN+25]. ScholarCopilot also enables user interaction during generation [WMN+25] by allowing the user to rewrite the generated text or forcefully

trigger citation retrieval between the consecutive generation steps but this feature is beyond the scope of this thesis and is thus not utilized for the experiments. ...

TODO

3. Methods

TODO

3.1. Retrieval-Augmented Generation

The RAG architecture [LPP+20] is comprised of a retriever p_η consisting of a query encoder q and a retrieval corpus d, and a generator p_θ (see Figure 3.1). Given a query x, the retriever $p_\eta(z|x)$ (with parameters η) computes a distribution over the text documents z in the retrieval corpus d according to x [LPP+20]. This distribution is used to obtain the top-k most relevant documents with respect to the query. The generator $p_\theta(y_i|x, z, y_{1:i-1})$, parametrized by θ, then predicts each next token y_i based on the previous tokens $y_{1:i-1}$, the given query x and a document z from the top-k retrieved documents (according to p_η) [LPP+20]. Lewis et al. base their retriever on the `Dense Passage Retriever` [KOM+20] and use a pre-trained `BART-large` [LLG+19] as the generator.

3.2. Dataset

This thesis evaluates the retrieval performance and generation quality of ScholarCopilot (SC, see section 3.3) with and without the passage retrieval module (PR, see section 3.4). ScholarCopilot retains its original retrieval corpus, while the retrieval corpus for the PR module uses the HDT dataset [HFB+24], since it requires fulltext data and the SC retrieval corpus only consists of abstracts. The PR module constructs the passages for each individual document during inference. The datasets used for evaluation are constructed from the HDT dataset as well.

The datasets for retrieval evaluation (section 4.2) each use a subset of the HDT dataset. Each test sample in the dataset consists of a citing text span and the corresponding cited document. All citing spans are extracted from the documents in the HDT dataset, provided the corresponding cited document appears in the ScholarCopilot retrieval corpus. This results in a total of 5.16M citing contexts. Following the evaluation setup in [WMN+25], all tokens after the citation are removed. This emulates the context the model would receive during generation, where it also only has access to the left-hand context of each `[RET]` token (see section 3.3, particularly the definition of $\hat{c}_j$ following Eq. 3.2). The citing context is grouped into separate datasets based on which section

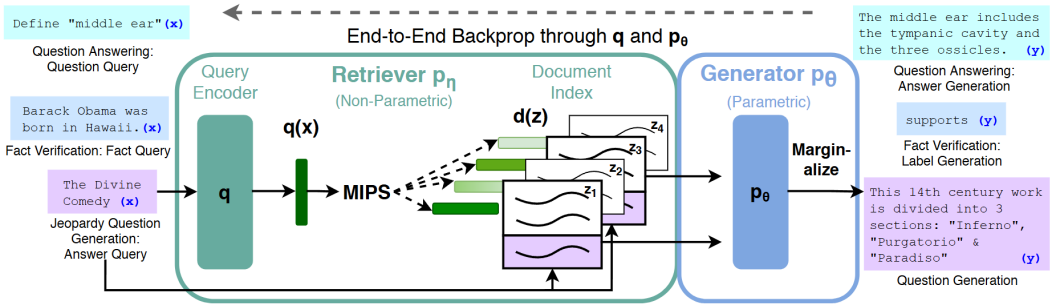

Figure 3.1.: Overview of the RAG architecture initially proposed by [LPP+20]. For finding the top-k documents Z', the authors used Maximum Inner Product Search (MIPS).

in the source document it is drawn from. This results in four datasets comprised of context from the sections `IntroductionAndRelatedWork` (4.4M samples), `Methods` (124k samples), `Experiments` (497k samples) and `Conclusion` (127k samples). The candidate sections in the HDT dataset for each of these datasets are determined via exact token matching, e.g. if the lowercase title of a section contains the string "methods", it is considered a methods section and therefor used to extract samples for the `Methods` evaluation dataset. Note that the dataset `IntroductionAndRelatedWork` combines two kinds of sections commonly found in scientific documents, this is done to imitate the ScholarCopilot evaluation dataset that includes only these sections. ScholarCopilot should yield the best performance on this dataset since the model is trained exclusively on titles, abstracts, introductions and related work sections [WMN+25].

The dataset for the generation evaluation (section 4.3) also uses the HDT dataset. Specifically, each of the 347k samples in the dataset consists of the title, abstract and the first two and a half sentences of the introduction (the HDT dataset already provides sentences, the half-sentence is obtained by tokenizing the source sentence using the ScholarCopilot tokenizer and truncating it to its left-hand half based its total number of tokens). This setup mimics the examples for initial generation contexts[1] provided with the official ScholarCopilot implementation. Before being used for evaluation, all samples are sorted by the arXiv id of the paper they were obtained from, which effectively also sorts them by their month of publication (see [sta25b]), and only the 100k most recently published papers are retained. This is done to ensure that ScholarCopilot predominantly cites papers published after the paper it is prompted to generate (that is, the paper that is used as the initial generation context).

[1] `https://github.com/TIGER-AI-Lab/ScholarCopilot/tree/main/run_demo/src`

3.3. ScholarCopilot

Instead of treating retrieval and generation as separate processing stages (see section 1) ScholarCopilot [WMN+25] integrates the information retrieval directly into the generation process. The model is trained to generate *retrieval tokens* (`[RET]`), that upon being generated, cause the model to interupt the generation process to retrieve a reference from the retrieval corpus [WMN+25]. The ScholarCopilot retrieval corpus[2] consists of abstracts of scientific papers, which are encoded by the ScholarCopilot model prior to training/inference and stored as a HNSW [MY18] index using Faiss [DGD+24] for efficient retrieval. Standard ScholarCopilot uses the retrieved abstract to inform its subsequent generation by replacing the `[RET]` tokens in the generated context with the corresponding abstracts.

For this thesis, I evaluated the official ScholarCopilot implementation[3]. In their paper, the authors state that ScholarCopilot can also integrate key excerpts[4] [WMN+25] but the official implementation only uses abstracts and provides no logic for extracting these excerpts. Because of that, I further on use the term "ScholarCopilot" to denote a version of this model that represents papers by their abstracts and does not use potentially more informative key excerpts.

ScholarCopilot is intended to assist with academic writing by providing "[...] text completion and citation suggestions" (see the official demo[5,6] for an example). In the following we will thus assume that the model is used to generate a scientific paper with citations. Let

$$Y^{\mathtt{in}} = \left(y_1^{\mathtt{in}}, y_2^{\mathtt{in}}, ..., y_l^{\mathtt{in}}\right) \tag{3.1}$$

denote the initial context given to the model. Since we seek to generate a scientific document, this context may consist of the title and abstract of the paper the model is supposed to generate. Let furthermore

$$Y_{1:i-1}^{\mathtt{gen}} = \left(y_1^{\mathtt{gen}}, y_2^{\mathtt{gen}}, ..., y_{j-1}^{\mathtt{gen}}, c_j, y_{j+1}^{\mathtt{gen}}, ..., y_{i-1}^{\mathtt{gen}}\right) \tag{3.2}$$

denote a token sequence generated by the model where c_j signifies that a `[RET]` token was generated by the model at generation step j. Note that the model may generate the retrieval token at multiple steps, and that the number of generated retrieval tokens is equal to the number of citations in the generated paper. The representation $\hat{c}_j$ of a `[RET]` token c_j computed by the model serves as an aggregated representation of the token sequence generated up to the `[RET]` token (e.g. $Y_{1:j-1}^{\mathtt{gen}}$

[2] `https://huggingface.co/datasets/TIGER-Lab/ScholarCopilot-Data-v1`

[3] See `https://github.com/TIGER-AI-Lab/ScholarCopilot`. I cloned the repository on the 13th of June 2025 and evaluated that version, which corresponds to commit `f8e6de9` in the source repository. As of the time of writing this (7th of September), only the README.md was extended.

[4] In the paper it says "key excepts", likely a typo.

[5] `https://huggingface.co/spaces/TIGER-Lab/ScholarCopilot`

[6] `https://www.youtube.com/watch?v=QlY7S52sWDA`

in Eq. 3.2), in the same manner as BERT [DCLT19] (and also for example CF-ViT [CLL+23]) uses the `[cls]` token as a representation of its input sequence. Thus $\hat{c}_j$ is unique for every j since it is dependent of the corresponding left-hand context.

When the ScholarCopilot generator p_G generates a retrieval token (that is, $p_G(*|Y^{\texttt{in}} Y^{\texttt{gen}}_{1:i-1})$ is maximal for the `[RET]` token, thus $* \leftarrow c_i$), the generation process is interrupted and instead, the representation $\hat{c}_i$ is passed to the retriever $p_R(z_i|\hat{c}_i)$ used to retrieve an encoded reference z_i corresponding to an abstract

$$Y^{\texttt{ref}}_i = \left(y^{\texttt{ref}}_{i,1}, y^{\texttt{ref}}_{i,2}, ..., y^{\texttt{ref}}_{i,k}\right) \tag{3.3}$$

from the retrieval corpus. Before passing the newly generated context to the generator the retrieval token c_i is replaced with the corresponding abstract to obtain an augmented generation output

$$\mathbb{Y}^{\texttt{gen}}_{1:i} = \left(y^{\texttt{gen}}_1, y^{\texttt{gen}}_2, ..., y^{\texttt{gen}}_{i-1}, y^{\texttt{ref}}_{i,1}, y^{\texttt{ref}}_{i,2}, ..., y^{\texttt{ref}}_{i,k}\right) \tag{3.4}$$

that is used to inform the sebsequent generation steps with the retrieved information. The generator $p_G(y_{i+1}|Y^{\texttt{in}} \mathbb{Y}^{\texttt{gen}}_{1:i})$ predicts the next token y_{i+1} at generation step $i+1$ based on the given initial context $Y^{\texttt{in}}$ and the augmented output $\mathbb{Y}^{\texttt{gen}}_{1:i}$. See Algo. 1 for a detailed overview of one generation step. Before presenting the generated sequence to the user, each inserted abstract is replaced with a token referencing the cited paper (when using latex formatting, this would for example be a citation like "\cite{vaswani2017attention}"). The user is presented with this modified sequence, as well as a list of all cited references.

3.4. Passage Retrieval

As outlined in section 3.3, ScholarCopilot (SC) represents each scientific document in the retrieval corpus with its abstract. But, depending on the context for which the model is supposed to retrieve a reference, the abstract might not be the most informative passage of a candidate paper (see section 1). Thus always using a paper's abstract as reference may add irrelevant information into the generated sequence which can negatively affect the model's reasoning capabilities [SCM+23].

This thesis introduces a *passage retrieval* module (PR) that is supposed to enhance the citation accuracy and generation quality of ScholarCopilot (see section 1). The module is agnostic to the RAG model, which makes it possible to use with any RAG architecture. However, this thesis only investigates how it affects ScholarCopilot. First, the module takes the top-k_{SC} documents

$$Z_i = \left(D_1, D_2, ..., D_{k_{SC}}\right) \tag{3.5}$$

returned by the SC retriever for a previously generated sequence $Y^{\texttt{gen}}_{1:i-1}$ and subdivides these documents into n-sized passages to obtain a set of candidate passages P_i.

Algorithm 1: Overview of one generation step of ScholarCopilot. Here Y denotes the vocabulary (the set of all tokens), Z the retrieval corpus (all encoded abstracts), G ScholarCopilot's decoder and getAbstract() a function that returns the unencoded abstract corresponding to z_i

Input: The initial context $Y^{\texttt{in}}$ and the previously generated sequence $Y^{\texttt{gen}}_{1:i-1}$
Output: The initial context $Y^{\texttt{in}}$ and the new generated sequence $Y^{\texttt{gen}}_{1:i}$

Algorithm: ScholarCopilotGenerationStep($Y^{\texttt{in}}, Y^{\texttt{gen}}_{1:i-1}$)
$y_i \leftarrow \arg\max_{y_i \in Y} p_G(y_i | Y^{\texttt{in}} Y^{\texttt{gen}}_{1:i-1})$
if y_i = *[RET]* **then**
  $\hat{c}_i \leftarrow G(y_i)$
  $z_i \leftarrow \arg\max_{z_i \in Z} p_R(z_i | \hat{c}_i)$
  $Y^{\texttt{ref}}_i \leftarrow$ getAbstract(z_i)
  $\mathbb{Y}^{\texttt{gen}}_{1:i} \leftarrow Y^{\texttt{gen}}_{1:i-1} Y^{\texttt{ref}}_i$
  return Y^{in}, $\mathbb{Y}^{gen}_{1:i}$
end
else
  $Y^{\texttt{gen}}_{1:i} \leftarrow Y^{\texttt{gen}}_{1:i-1} y_i$
  return Y^{in}, $Y^{gen}_{1:i}$
end

Prior work on understanding and utilizing discourse structure of prose has shown that the grammatical structure across multiple sentences can inform the reader of what is important in a document [Rum75] and can guide comprehension and recall [Man78]. Since the PR retrieval corpus (see section 3.2) consists of documents published on arXiv[7] and thus they are curated and "[checked] for scholarly value" [sta25a], it is reasonable to assume that these documents are already well-formed and have meaningful semantic structure spanning multiple sentences and paragraphs. To preserve this structure of each document D_i in the passages obtained from it as much as possible, each document D_i is first split into sections

$$D_i = \left(\mathcal{E}_1, \mathcal{E}_2, ..., \mathcal{E}_{|D_i|}\right) \tag{3.6}$$

by utilizing the fact that the HDT dataset [HFB+24] (see also section 3.2) already provides its documents structured into sections. Since the HDT dataset is comprised of computer science papers, many of these sections such as for example Introduction, Methods or Experiments typically share structural and semantic similarities across documents. Each section $\mathcal{E}_i$ is split into passages

$$\mathcal{E}_i = \left(\mathcal{P}_1, \mathcal{P}_2, ..., \mathcal{P}_{|\mathcal{E}_i|}\right) \tag{3.7}$$

[7] https://arxiv.org/

which are sequences of n or less tokens $\mathcal{P}_i = (t_1, t_2, ..., t_n)$ ($\mathcal{P}_{|\mathcal{E}_i|}$ might be shorter than n tokens since $|\mathcal{E}_i| \equiv 0 \mod n$ likely does not hold for every section in every document). This ensures that each passage only consists of sentences taken from a single section in the source document. All passages are collected into a corpus P_i and then ranked by a decoder model R_{PR} based on how relevant they are to the context $Y^{\texttt{gen}}_{1:i-1}$ (assuming y_i denotes the `[RET]` token that triggered the retrieval, see section 3.3, particularly Algo. 1). The passage retrieval prompt that is used to instruct the decoder is provided in section A.2. To achieve reliably reproducable relevance assessment performance the scores computed by R_{PR} are aggregated using Batched Self-Consistency [KDSR25]. Furthermore, to reduce the computational burden, the context is truncated to size w – yielding $Y^{\texttt{gen}}_{i-w-1:i-1}$ as the new context – before being passed to R_{PR}. I found that passing longform context tends to worsen the performance of the passage ranking model. The top-ranked passage is then integrated into the generated text in the same manner as the abstract would have been integrated when using standard ScholarCopilot (see Eq. 3.4). How the PR module is integrated into ScholarCopilot is shown in Algo. 2.

It is important to point out that the performance of the PR module is partially dependent on the performance of ScholarCopilot, since only the top-k_{SC} documents of ScholarCopilot are passed to the PR module (for the experiments I use $k_{SC} = 2k$ when evaluating with Recall@k). For Recall@10 for example, it is impossible for the correct document to be chosen by the PR module if it is not in the top 20 documents ranked by ScholarCopilot. This effect can be mitigated by choosing a larger k_{SC}, but this would also drastically increase the effective runtime of the PR module, since the module computes a score for each of the p passages obtained from each of the k_{SC} documents m times (where m is the number of LLM calls for the Batched Self-Consistency score aggregation, see [KDSR25]).

The decision to let the ScholarCopilot retriever first retrieve abstracts and not directly passages is motivated by the success of employing mixed-resolution tokenization [CLL+23, HRBB23] for image processing with the Vision Transformer [DBK+20]. In [CLL+23, HRBB23], mixed-scale tokenization is primarily used to decrease the length of the input sequence to alleviate the computational burden imposed by the Transformer backbone whose attention layer has $O(n^2)$ time complexity (see [VSP+17]). Both approaches first split a given image into larger *coarse-scale* patches and then identify the most informative k patches which are then further subdivided into four *fine-scale* tokens before the image, now represented by a mixture of coarse and fine-scale tokens, is passed to the ViT backbone. ScholarCopilot and the PR retriever use the same backbone architecture (see section 4) for retrieval, so if we consider the abstract of a document D_i its coarse-scale representation and its $|D_i|$ passages (for example, when assuming 8192 tokens per document and a passage length of 512 this would mean $|D_i| = 16$) to be the fine-scale representation, then the PR module implements a similar two-stage *mixed-scale retrieval*. Let C denote the retrieval corpus and for simplicity lets assume that all documents $D_i \in C$ have the same number of $|D_i|$ passages. Then performing passage retrieval on the set of all

passages would mean comparing one query to $|C| \cdot |D_i|$ passages. Using mixed-scale retrieval requires $|C| + k_{SC}|D_i|$ comparisons, a lot less when k_{SC} is chosen significantly smaller that $|C|$ (which is reasonable to assume and is also the case for my experiments, see section A.1). This effect is further amplified when using Batched Self-Consistency.

The abstract of a scientific document typically presents the reader with knowlege of its content, can arouse interest and can indicate whether the fulltext or part of it merits further attention [SM90]. It can serve as a condensed representation of the whole document [CO06]. An abstract can thus also be considered a semantic coarse-scale representation of a document since it usually contains short general statements about the topics discussed in the document. For example, the sentence "We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely." from the abstract of [VSP+17] conveys the information that the Transformer is based on attention but does not elaborate on how attention works.

On the other hand, the passages obtained from a document's sections often contain more specific information. Consider for example "We call our particular attention 'Scaled Dot-Product Attention' [...]. The input consists of queries and keys of dimension [...]" (the beginning of section 3.2.1 in [VSP+17]) which leads into a detailed explanation of the attention layer. Passages can thus similarly be considered *fine-scale* in a semantic sense, which allows the PR retriever to better distinguish between different aspects of the topic discussed in each paper than ScholarCopilot (that relies on abstracts alone). This could potentially result in increased retrieval performance, especially if the PR retriever is specifically trained with a contrastive loss function to identify relevant passages. However, this remains an assumption for now because training the retriever is beyond the scope of this thesis and is thus left for future research (see also section 5).

Algorithm 2: Overview of one generation step of ScholarCopilot with Passage Retrieval. getDocument() denotes a function that returns the fulltext paper corresponding to z_j

Input: The initial context $Y^{\texttt{in}}$ and the previously generated sequence $Y^{\texttt{gen}}_{1:i-1}$
Output: The initial context $Y^{\texttt{in}}$ and the new generated sequence $Y^{\texttt{gen}}_{1:i}$

Algorithm: ScholarCopilotWithPRGenerationStep($Y^{\texttt{in}}, Y^{\texttt{gen}}_{1:i-1}$)
$y_i \leftarrow \arg\max_{y_i \in Y} p_G(y_i | Y^{\texttt{in}} Y^{\texttt{gen}}_{1:i-1})$
if y_i = *[RET]* **then**
  $\hat{c}_i \leftarrow G(y_i)$
  $Z_i \leftarrow \emptyset$
  $Z' \leftarrow Z$
  while $|Z_i| < k_{SC}$ **do**
    $z_j \leftarrow \arg\max_{z_j \in Z'} p_R(z_j | \hat{c}_i)$
    $Z_i \leftarrow Z_i \cup \{z_j\}$
    $Z' \leftarrow Z' \setminus \{z_j\}$
  end
  $P_i \leftarrow \emptyset$
  foreach $z_j \in Z_i$ **do**
    $D_j \leftarrow$ getDocument(z_j)
    foreach $\mathcal{P}_k \in D_j$ **do**
      $P_i \leftarrow P_i \cup \mathcal{P}_k$ // see Eq. 3.6 and Eq. 3.7
    end
  end
  $Y^{\texttt{ref}}_i \leftarrow \arg\max_{\mathcal{P}_k \in P_i} p_{R_{PR}}\left(\mathcal{P}_k | Y^{\texttt{gen}}_{i-w-1:i-1}\right)$
  $\mathbb{Y}^{\texttt{gen}}_{1:i} \leftarrow Y^{\texttt{gen}}_{1:i-1} Y^{\texttt{ref}}_i$
  return Y^{in}, $\mathbb{Y}^{gen}_{1:i}$
end
else
  $Y^{\texttt{gen}}_{1:i} \leftarrow Y^{\texttt{gen}}_{1:i-1} y_i$
  return Y^{in}, $Y^{gen}_{1:i}$
end

4. Experiments

TODO

4.1. Evaluation Methodology

The main focus of this thesis is it to determine how the use of the passage retrieval module affects the retrieval accuracy and generation quality of ScholarCopilot. ScholarCopilot retains its original backbone model `Qwen2.5-7B` [YYZ+24]. Since the retriever of the passage retrieval module is implemented with Batched Self-Consistency, it is required to follow multiple permutations of the same instruction during inference reliably and consistently (see [KDSR25]). Instruction tuned models, that is, LLMs that are fine-tuned on (`instruction`, `desired-output`) pairs, are more successful at following instructions closely rather than just minimizing the next token prediction error [ZDL+25]. To make use of this, the PR module uses the instruction-tuned variant of the Qwen2.5 model, `Qwen2.5-7B-Instruct` [YYZ+24], as the passage retriever. Even though it has been shown that instruction tuned models may not fully utilize given instructions and are instead just good at learning the specified output format [KP23], this ability alone is valuable enough for the automated parsing of an LLMs' response to justify using an instruction tuned model (especially when using Batched Self-Consistency which greatly increases the number of responses that need to be parsed).

With ScholarCopilot as baseline, I evaluated the impact of using the PR module on the retrieval performance for a variety of retrieval contexts. To this end, I introduce the *single-step retrieval* task which aims to assess the models accuracy when resolving a single `[RET]` token during generation. This task closely follows the evaluation methodology used by [WMN+25]. For each of the section groups (`IntroductionAndRelatedWork`, `Methods`, `Experiments`, and `Conclusion`), 1k randomly selected samples are used for the evaluation. Single-step retrieval is evaluated using the Recall@k metric, which is given as the fraction of cases where the ground truth document appears in the top-k retrieved documents [WMN+25]. As outlined in section 3.2, the model is given the left-hand context of a citation drawn from an existing paper, together with the cited document as the ground truth. A retrieval token is appended to this context before the resulting sequence is passed to ScholarCopilot, causing the model to compute a representation of the `[RET]` token based on the given context. This representation is then used to compute a distribution over

the ScholarCopilot retrieval corpus that measures the relevance of each document in the corpus to the given context. When evaluating standart ScholarCopilot, the k most relevant documents according to this distribution are compared to the ground truth, otherwise the top-k_{SC} most relevant documents are passed to the passage retrieval module (for the reported results I use $k_{SC} = 2k$, see also section A.1). For each of the top-k ranked passages the PR module returns the document from which each passage was taken. These k documents are then compared to the provided ground truth. Additionally, I evaluated two different strategies for replacing other citations that appear in the citing context. Other citations are either (1) masked using a `[MASK]` token, this is also how [WMN+25] handle citations, or (2) replaced with the abstract of the cited paper, which is done to imitate the generated context the retriever would be given during inference when the model encounters a `[RET]` token. The datasets that use these replacement strategies are denoted as `masked` and `with-abstracts` respectively. Following the setup in [WMN+25], the given citing context is not truncated when passed to ScholarCopilot.[1] However, I found that passing the entire context to the PR module reduces performance, thus only the last w tokens of the citing context are used when the PR retriever is prompted to rank a given set of passages. See section A.2 for the detailed passage retrieval instruction.

Since using human experts to evaluate the quality of LLM answers is costly and slow [HPS+25], the quality of the generated academic text is evaluated using `DeepSeek-R1-Distill-Qwen-7B` [DAGY+25]. Following the setup in [WMN+25], the evaluating model (judge) is instructed to compare academic text generated by either ScholarCopilot or ScholarCopilot with passage retrieval to the ground truth document (which is the beginning of a scientific paper from arXiv, see section 3.2). It is instructed to evaluate the given generated text according to the five dimensions `Content Relevance`, `Logical Coherence`, `Academic Rigor`, `Background Completeness` and `Innovation Statement`, see section A.3 for the detailed instruction.[2] To assess the reliability of the judge, the evaluation is performed two times, once by (1) instructing the judge to only provide a score from 0 to 5 for each evaluation dimension and (2) by additionally prompting the judge to provide reasoning for its decision before assigning each score. To achive consistent and reliable scoring, the scores for the evaluation dimensions are aggregated using Batched Self-Consistency. Specifically, the order in which the individual evaluation dimensions appear in the generation evaluation prompt is randomized every time the judge is instructed to compute a ranking and for each generated paper the judge is queried to evaluate it $m_{\texttt{judge}}$ times. The evaluation is performed on 100 ground truth samples which are selected randomly from the generation evaluation dataset. To save time and computational

[1] [WMN+25] only use the last sentence of the context when evaluating their baseline models (`BM25` and `E5-Mistral-7B-Instruct`) but use the full context when evaluating ScholarCopilot.

[2] It is intentional that I don't go into detail on how these categories should be interpreted because this information is also not given in the judge instruction. How the judge interpretes the evaluation categories is thus entirely dependent on the pre-training and fine-tuning data of Qwen2.5 and Deepseek-R1 respectively. This design follows [WMN+25] who also do not provide their judge with interpretation suggestions.

Model	Dataset	intro+relwork	methods	experiments	conclusion	Σ
SC	masked	−0.06	−0.04	−0.09	−0.07	−0.26
SC	with-abstracts	−0.05	−0.06	−0.07	−0.06	−0.24
SC+PR	masked	−0.03	−0.03	−0.05	−0.05	**-0.16**
SC+PR	with-abstracts	−0.04	−0.04	−0.03	−0.04	**-0.15**

Table 4.1.: Cumulative Recall@k difference from $k = 10$ over $k = 5$ to $k = 1$.

resources, the text generation (by the evaluated models) is terminated early if the number of generated tokens exceeds 30k (this limit includes the abstracts/passages inserted during generation).

4.2. Single-Step Retrieval

Citation Recall. The recall results of the single-step retrieval evaluation are visualized in Figure 4.1 and Figure 4.2. Using the passage retrieval module consistently worsens the performance of ScholarCopilot (on average −0.035%/−0.027% recall on the `masked`/`with-abstracts` datasets). However, as shown in Table 4.1, when using the PR module the accumulated loss in recall from $k = 10$ to $k = 1$ is much lower compared to only using ScholarCopilot. So if given the correct document, the PR module tends to push the correct document further towards the top of the relevance ranking in comparison to the ranking computed by ScholarCopilot. This suggests that utilizing passage retrieval can improve scientific citation retrieval compared to doing retrieval solely based on abstracts.

Reproducability. ScholarCopilot alone is unable to reproduce the retrieval performance reported in [WMN+25] (which is 64.8%/40.1% for Recall@10 and Recall@1 respectively). The authors do not explicitly state this in their paper but it seems they only used Introduction and Related Work sections to obtain the citing contexts for their evaluation dataset.[3] Thus, using the `IntroductionAndRelatedWork` dataset with `masked` citations (see Figure 4.1 on the left) as the evaluation setup replicates their methodology (see [WMN+25, 4.2]) and should yield a performance similar to what is stated in [WMN+25]. A possible cause for the lower recall on my dataset may be that ScholarCopilot overfits to their dataset. What exactly is responsible for this significant deviation in performance is left for future research.

Recall Performance Analysis. ScholarCopilot does not yield the best performance on contexts from Introduction and Related Work sections (which is what it was trained on) but instead shows significantly better performance on Experiment sections. This

[3] The file `scholar_copilot_eval_data_1k.json` which is downloaded from `https://huggingface.co/datasets/ubowang/ScholarCopilot-TrainingData` when executing the official ScholarCopilot implementation – and I assume is what they used for their evaluation – only includes Introduction and Related Work sections.

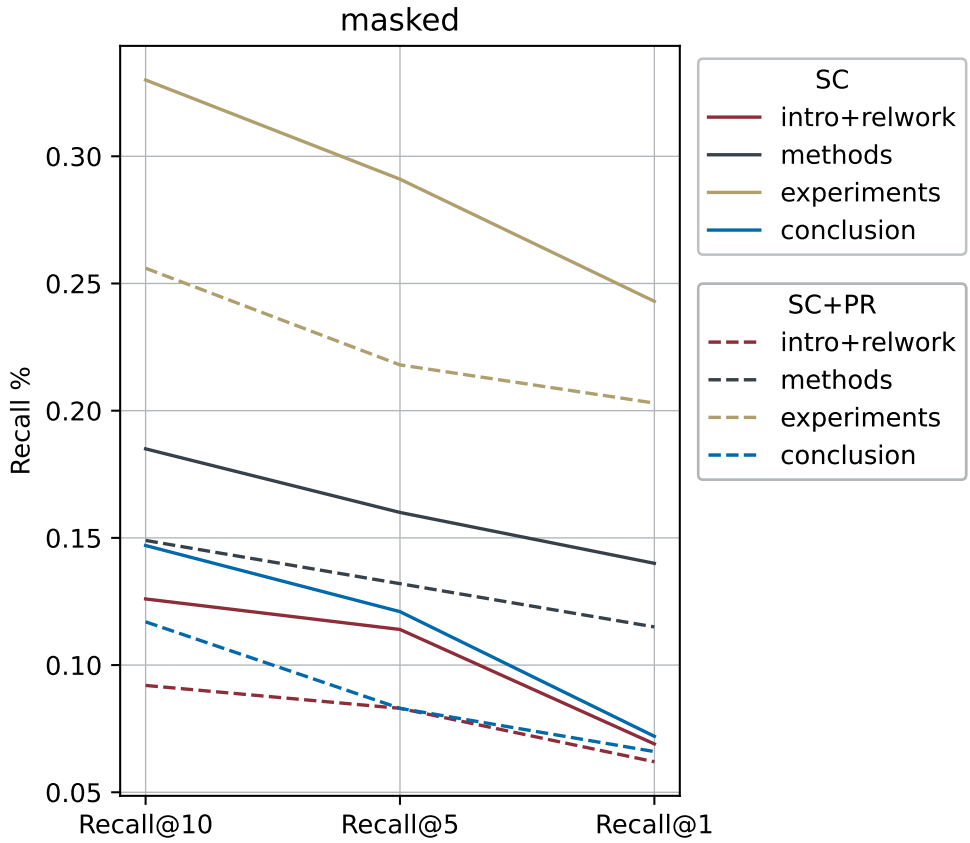

Figure 4.1.: Comparison of single-step retrieval performance between ScholarCopilot (SC) with and without passage retrieval (PR) on the `masked` dataset. `intro+relwork` denotes context drawn from Introduction and Related Work sections.

is possibly due to the fact that Experiments sections often include citations in a form similar to for example "Documents are tokenized with the same byte-pair encoding as GPT-2 (Radford et al., 2019)."[4] where the citation is directly preceded by the name of a model, specific algorithm or otherwise meaningful entity and the cited document is the paper where this algorithm was first proposed. This makes it easier for the model to identify relevant documents: taking again the previous example, a document that is about GPT-2 and would thus be a good candidate to cite after a sequence like "Documents are tokenized with the same byte-pair encoding as GPT-2" will likely frequently use the term "GPT-2". ScholarCopilot in particular benefits from this since it only retrieves from a corpus of abstracts and an abstract typically includes the name of the algorithm/method proposed in a paper while

[4]This example is taken from [LLG+19]

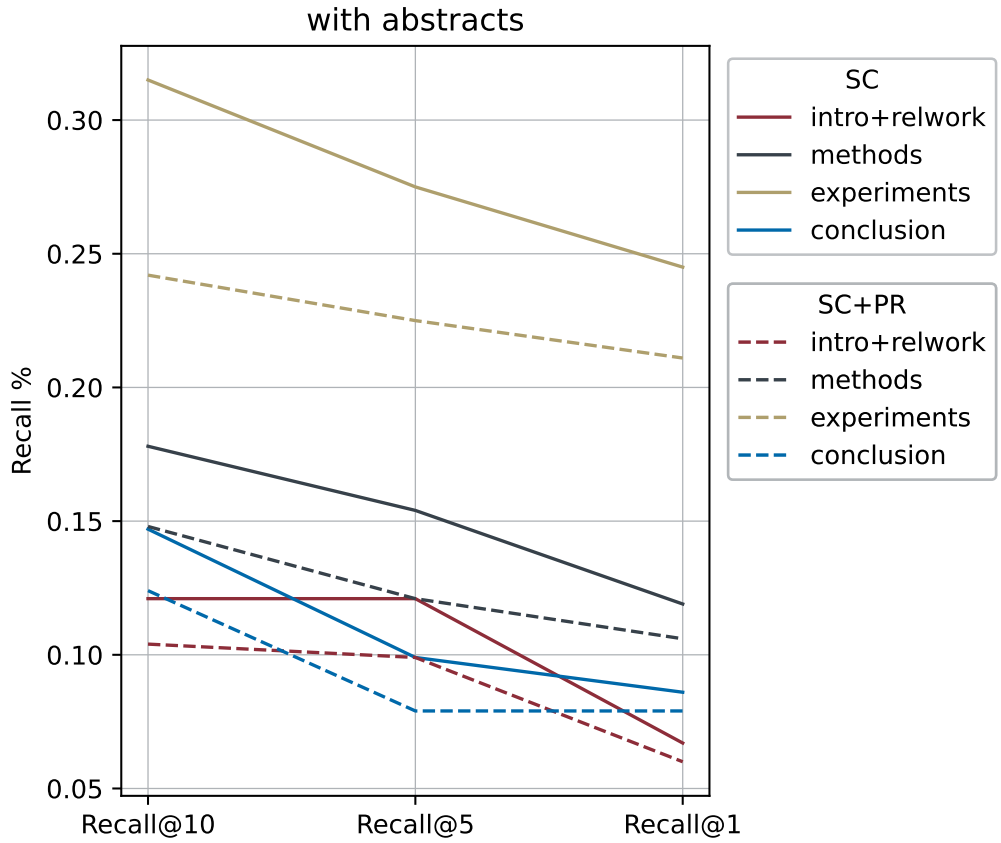

Figure 4.2.: Comparison of single-step retrieval performance between ScholarCopilot (SC) with and without passage retrieval (PR) on the `with-abstracts` dataset.

simultaneously being less likely than for example the Introduction to name methods that are only similar to or preceding the proposed method. As shown in Table 4.2, the `Experiments` dataset has indeed the highest percentage of cases where the token before the citation appears in the abstract of the cited paper. Note that this metric is likely influenced by some false positives where the last token is not a semantically meaningful entity but rather just a token that appears often in scientific abstracts.

Correct Retrieval Overlap. As shown in Table 4.3, the PR module sometimes retrieves the correct document despite ScholarCopilot being previously incorrect, especially for low k (when using Recall@k as measure for correctness). This suggests that the PR module has the capacity to further improve citation retrieval compared to only utilizing ScholarCopilot. The fact that the performance improvement is more prevalent for small k (and consequently small k_{SC} which directly influences the number of candidate passages) also indicates that the PR module may yield

	intro+relwork	methods	experiments	conclusion
last token in abstract	0.08	0.12	**0.22**	0.11

Table 4.2.: Fraction of cases where the last token before the citation appears in the abstract of the cited document.

better performance if given fewer passages. To illustrate the how retrieving passages instead of abstracts can be beneficial for generative academic writing, consider the following example where ScholarCopilot fails to retrieve the correct document while the PR module is able to retrieve a passage from the correct document. Note that the following examples are slightly shortened, refer to section A.4 for the full text. The given context for retrieval was taken from [QBK+21]:[5]

```
\textbf{Retrieval context}

1 Introduction Task-oriented dialog (TOD) systems have been widely
  deployed for popular virtual assistants, like Siri and Google
  Assistant. They help people with tasks such as booking restaurants and
  looking for a hotel by searching databases with constraints provided by
  users. After retrieving a result from the database, a system may
  continue by conducting actions like making a reservation or providing
  more information about receiving the result. However, there can be
  multiple results from the database that match the same constraints. For
  example, as shown in Fig.1, the system finds two available hotels at
  different locations when the user is asking the system to help book a
  hotel. This kind of ambiguity stops system from proceeding until the
  system finds out which result the user looks for. [...] Our work mainly
  focuses on improving models ability of understanding the answers by
  augmenting two existing task-oriented dialog datasets: MultiWOZ
  <|CIT-MASK|> and Schema-Guided Dataset (SGD) #ref25#. [...] To
  strengthen the model with the ability to handle the ambiguity, we
  propose to augment these two datasets with disambiguation turns, where
  the system provides all possible matched results and lets the user make
  their own decision based on the complete information. Specifically, we
  first extract templates from the SIMMC 2.0 dataset <|CIT-MASK|>
```

The citation for which the model is supposed to retrieve a reference is the last <|CIT-MASK|> token. The context immediately preceeding the citations implies that the target reference is the publication in which the SIMMC 2.0 dataset is proposed (which would be [KMGD21]). It is further implied that this dataset is commonly used in the context of either search query disambiguation or task-oriented dialog. Given this context, ScholarCopilot retrieved the following abstract from [LH21]:

[5] "#refXX#" are citation markers from the HDT dataset the preprocessing failed to mask properly. However, the markers do not carry any meaningful information like authors, publication date or part of the paper title which they typically would include in standard LaTeX source code. Thus they are effectively already "masked" and this bug should have no significant impact on the model performance.

```
\textbf{Retrieved reference (SC)}

Abstract This paper presents our work on the Situated Interactive
  MultiModal Conversations 2.0 challenge held at Dialog State Tracking
  Challenge 10. SIMMC 2.0 includes 4 subtasks, and we introduce our
  multimodal approaches for the subtask #1, #2 and the generation of
  subtask #4. SIMMC 2.0 dataset is a multimodal dataset containing image
  and text information, which is more challenging than the problem of
  only text-based conversations because it must be solved by
  understanding the relationship between image and text. Therefore, since
  there is a limit to solving only text models such as BERT or GPT2, we
  propose a multimodal model combining image and text. We first pretrain
  the multimodal model to understand the relationship between image and
  text, then finetune our model for each task. [...]
```

While this abstract, and consequently the paper it is drawn from, is indeed closely linked to the SIMMC 2.0 dataset, it is not the paper that introduced this dataset but instead proposes a method for solving one of the tasks formulated in the SIMMC 2.0 paper (see [LH21, KMGD21]). While the retrieved abstract does even briefly explain what this dataset is, it also contains a lot of information that is irrelevant for the given citing context (such as what models the authors use). ScholarCopilot fails to realize that [LH21] do not propose the SIMMC 2.0 dataset themselves but rather only make use of it. A good reference for the given citing context should not only explain what the SIMMC 2.0 dataset is but also contain as little unrelated information as possible. The passage retrieval module on the other hand manages to retrieve a passage from the correct document [KMGD21]:

```
\textbf{Retrieved reference (SC+PR)}

The aim of the SIMMC 2.0 dataset is to emulate futuristic, real-world
  shopping scenarios where humans converse with a dialog agent in natural
  language grounded in a situated multimodal context. As a step towards
  this intelligent conversational agent, we leverage the dialogs and
  annotations in our dataset and propose four benchmark tasks (summarized
  in Tab.4) along with evaluation metrics. These tasks capture several
  multimodal conversational reasoning challenges, as elaborated next. In
  a real-world conversation, humans often use shorthands (coreferences)
  in order to refer to objects / events that have already been mentioned
  in the dialog. While we reserve modeling coreference resolution as a
  challenging task in Sec.4.2, it is important for the system to
  recognize ambiguous uses of such coreferences even before attempting to
  resolve them. [...] The multimodal disambiguation task tests this
  ability of the agent. More concretely, given the dialog history and the
  current user utterance, multimodal disambiguation requires the agent to
  predict a binary label conditioned on the multimodal context, to
  indicate the presence of a referential ambiguity in the user utterance.
  [...] These mentions can be resolved through (1) the dialog context
```

Dataset	Metric	intro+relwork	methods	experiments	conclusion	μ
masked	Recall@10	0.91	0.97	0.98	0.91	0.943
masked	Recall@5	0.94	0.94	0.96	0.94	0.945
masked	Recall@1	**0.79**	0.89	**0.88**	0.88	**0.86**
with-abstracts	Recall@10	0.9	0.93	0.96	0.95	0.935
with-abstracts	Recall@5	0.86	0.97	0.95	0.89	0.918
with-abstracts	Recall@1	0.87	**0.87**	0.93	**0.84**	0.878

Table 4.3.: Fraction of correct retrievals by ScholarCopilot given correct retrieval by the PR module (such that the value would be 1 if SC was correct every time the PR module was correct and 0 if SC was incorrect whenever the PR module was correct). A retrieval result is considered correct, if it satisfies the given metric.

```
(e.g. A: This shirt comes in XL and is $29. #eq50# U: Please add it to
cart., or (2) the multimodal context (e.g. U: How much is that red
shirt? ), or (3) both (e.g. U: How much is the one next to the one you
mentioned?). [...]
```

The retrieved reference is a passage from the section that introduces the benchmark tasks proposed together with the dataset (see [KMGD21, 4]). Similar to the abstract ScholarCopilot selected as the reference for the given citing context, this passage includes information that is irrelevant (like cross-references to other parts of the paper) or too elaborate for a citation (e.g. the examples at the end), it does however provide additional relevant information. In particular, it states that SIMMC 2.0 "[emulates] futuristic, real-world shopping scenarios where humans converse with a dialog agent in natural language" which provides a possible explanation for why this dataset is useful for developing "task-oriented dialog [...] systems" but also mentions that one of the benchmark tasks is about query disambiguation, a problem that is also discussed in detail in the citing context. While the task of obtaining a concise reference by extracting only the most relevant information from a candidate passage is left for future research, this example does still highlight the potential that fine-grained passage retrieval has to identify more relevant references than what can be retrieved solely from a corpus of abstracts.

4.3. Generation Quality

Quality Comparison. The results designed to primarily assess the generation quality are shown in Figure 4.3. ScholarCopilot achieves higher scores in all evaluation categories, with a score of 15.5/25 across all dimensions. Using passage retrieval only yields a total score of 14.9/25 (-0.6), falling behind most in `Background Completeness` and `Academic Rigor`. This suggests that passage retrieval is not

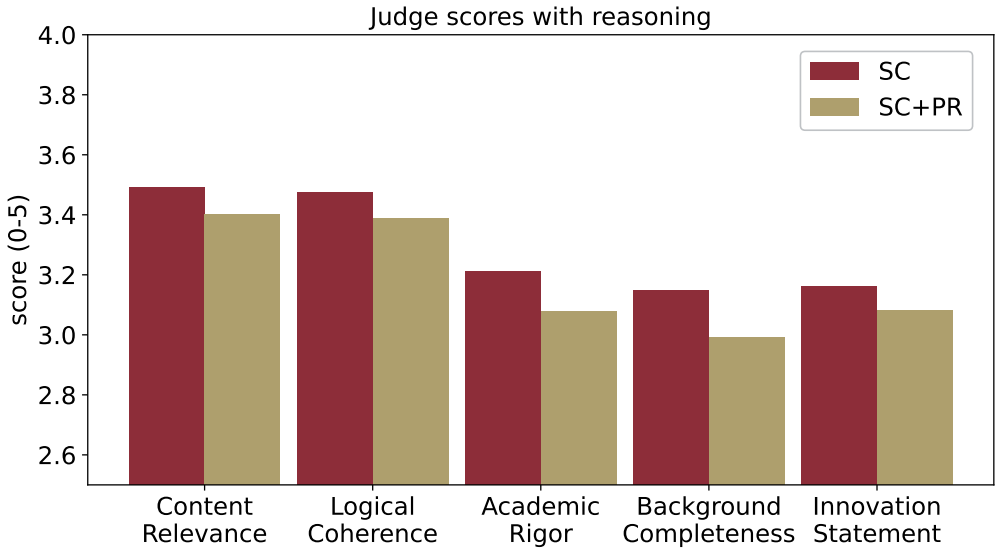

Figure 4.3.: Comparison of the generation quality scores between ScholarCopilot (SC) with and without passage retrieval (PR) when the judge is instructed to provide reasoning for its scoring.

suited to improve the generation quality of academic text generation with citations.

Evaluation Reliability. Instructing the judge to provide reasoning for its generation quality assessment yields ranking scores with a mean difference of 0.11 between the scores achieved with versus without passage retrieval. Using a prompt that does not instruct the model to reason about its decision results in a much smaller mean difference of 0.01 (-0.1, see also Figure 4.4). This makes it more difficult to identify key strengths and weaknesses of the evaluated methods. Furthermore, as shown in Figure 4.5 the standard deviation of the individual ranking scores for each evaluation dimension when instructing the judge $m_{\texttt{judge}}$ times to rank the same generated sequence without reasoning is with on average 2.35 significantly higher compared to the deviation of 2.14 (-0.21) with reasoning. These results indicate that the judge is more uncertain in its evaluation when not forced to reason about its scoring. See section A.5 for an example of an evaluation with reasoning generated by the judge. It is also important to note that the evaluation and consequently the scores computed by the judge don't always align well with human judgement, an example of this is shown in section A.6.

4.4. Additional Experiments

To leverage recent advances in information retrieval (IR), I conducted additional experiments using ReasonIR [SQK+25], a bi-encoder model developed for general-

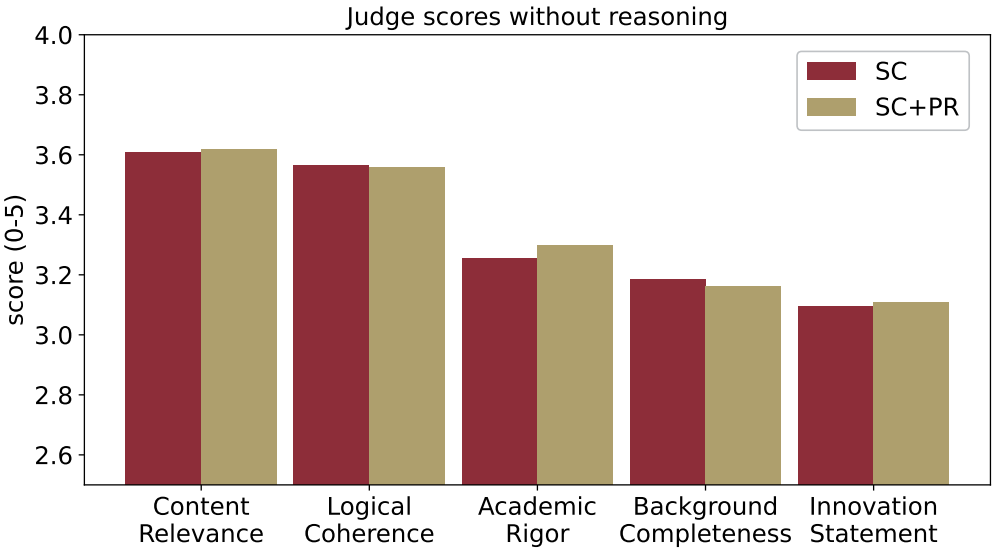

Figure 4.4.: Comparison of the generation quality scores between ScholarCopilot (SC) with and without passage retrieval (PR) when the judge is instructed to only provide scores.

purpose reasoning-intensive IR and RAG. `ReasonIR-8B` is a modified `Llama3.1-8B` [TLI+23] model that uses a bi-directional attention mask [MHW+24] instead of a causal attention mask [SQK+25]. It is trained using contrastive loss on a mixture of public datasets, as well as custom synthetic training data that is designed to improve the models' performance on reasoning-intensive tasks and long queries with 300-2000 tokens (see [SQK+25]). I tested using ReasonIR as part of the passage retrieval module, where it is primarily used to reduce the number of candidate passages passed to the passage retrieval module. Specifically, before the set of all candidate passages P_i (see Algo. 2) is passed to the passage retriever R_{PR}, the ReasonIR retriever $R_{\texttt{ReasonIR}}$ is used to rank all passages $\mathcal{P}_{i,j} \in P_i$ according to how relevant to the query $Y^{\text{gen}}_{i-w-1:i-1}$ they are (note that the context is truncated using a context window of size w, just like for R_{PR}). Out of the ranked passages, only the top $k_{\texttt{ReasonIR}}$ most relevant passages are retained and passed to R_{PR}. The rest of the PR module remains unchanged. Reducing the number of input passages to the passage retriever R_{PR} is designed to have two main effects: (1) as shown in section 4.2, reducing the number of candidate passages tends to improve the performance of R_{PR} and (2) does significanty reduce the high effective runtime added by aggregating the ranking scores of R_{PR} via Batched Self-Consistency (provided that the effective runtime of ReasonIR on the same amount of input data is lower than using R_{PR} with Batched Self-Consistency, which was the case during my evaluation).

I evaluated the in this way modified PR module, here denoted as `PR_ReasonIR`, against ScholarCopilot on the single-step retrieval task using the dataset with `masked`

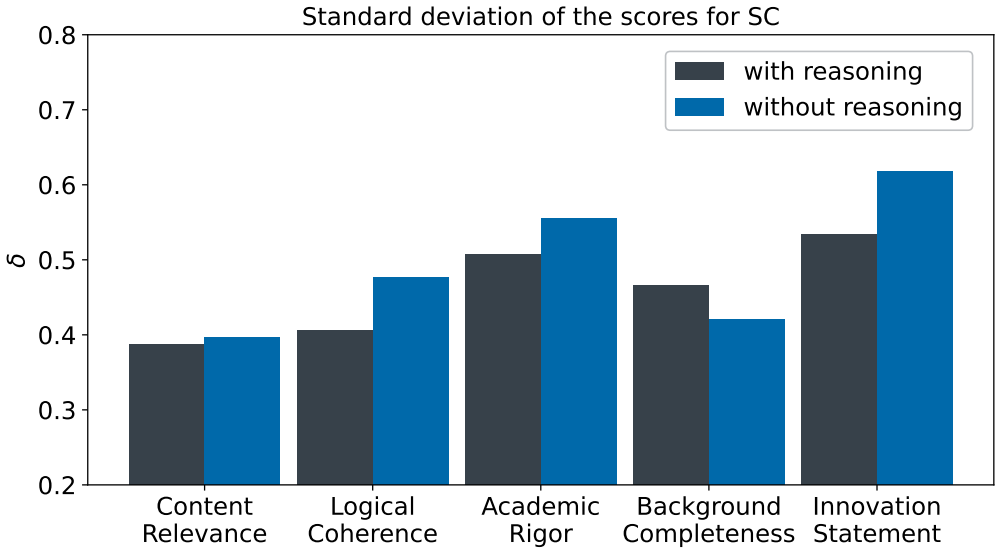

Figure 4.5.: Comparison of the standard deviation of the computed ranking scores for each evaluation dimension when the judge is instructed with versus without asking to provide reasoning.

citations. The evaluation was performed on 1k randomly selected samples per section group with Recall@10 as the evaluation metric. As the results visualized in Figure 4.6 show, using ReasonIR as an intermediate retrieval step worsens the performance of the PR module significantly. The investigation why this is the case is left for future work (mostly because this would have considerably lengthend the thesis). Nevertheless, it is likely still beneficial to look further into methods for reducing the number of passages passed to the PR module since that is the part responsibe for the majority of the effective runtime of the whole inference pipeline (due to the use of Batched Self-Consistency).

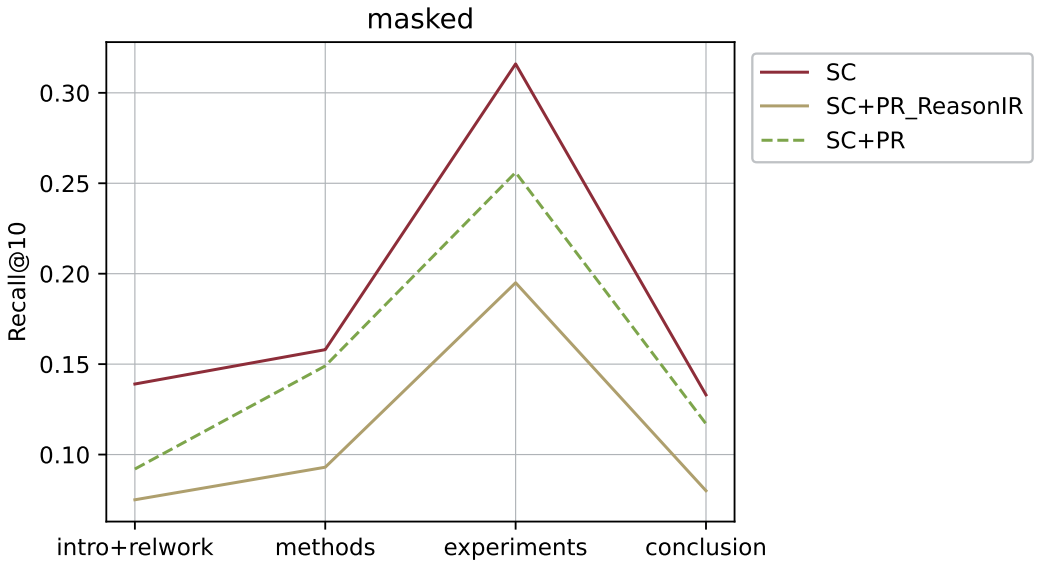

Figure 4.6.: Comparison of single-step retrieval performance between ScholarCopilot (`SC`) with passage retrieval (`PR`) and with PR with ReasonIR (`PR_ReasonIR`) on the `masked` dataset. Note that the results for `SC+PR` are on different evaluation samples and have their hyperparameters adjusted for PR with one retriever (such that in particular $k_{SC} = 2k = 20$ while $k_{SC} = 3k = 30$ for `PR_ReasonIR`, see section A.1 for details).

5. Limitations and Future Work

Passage retrieval in the form that is evaluated in this thesis is not an improvement over ScholarCopilot in either citation recall or generation quality. However, the evaluation results do suggest that passage retrieval has the capacity to enhance ScholarCopilot by being able to retrieve passages that are more informative that the abstracts retrieved by ScholarCopilot, particularly in cases where ScholarCopilot fails to retrieve the correct document. This additional information can potentially be used to inform the generation of academic writing, although according to the conducted LLM-based generation quality evaluation, this does currently not result in more coherent, rigorous, complete or innovative generation.

TODO

Evaluation methodology. TODO

Disadvantages of using passages. Passages can also, just like abstracts, introduce irrelevant or misleading information into the generation process. This is currently not addressed by the PR module which only subdivides candidate papers into evenly-sized passages. More fine-grained subdivision or summarizing passages before ranking might help to reduce the impact of irrelevant content in candidate passages on their ranking. Furthermore, in academic writing, papers are often cited because of their primary contributions (see e.g. section A.4). Taking the Transformer paper [VSP+17] as an example, let $\mathcal{P}_1$ and $\mathcal{P}_2$ be two passages drawn from [VSP+17] where $\mathcal{P}_1$ denotes the passage

```
Recurrent neural networks, long short-term memory [13] and gated
  recurrent [7] neural networks in particular, have been firmly
  established as state of the art approaches in sequence modeling and
  transduction problems such as language modeling and machine translation
  [35, 2, 5].
```

[VSP+17, 1] and $\mathcal{P}_2$ denotes the passage

```
To the best of our knowledge, however, the Transformer is the first
  transduction model relying entirely on self-attention to compute
  representations of its input and output without using sequence-aligned
  RNNs or convolution. In the following sections, we will describe the
  Transformer, motivate self-attention and discuss its advantages over
  models such as [17, 18] and [9].
```

[VSP+17, 2]. $\mathcal{P}_1$ is much less likely to be the reason why [VSP+17] is cited in a given context than $\mathcal{P}_2$ because the latter describes the method proposed in [VSP+17] whereas the former mentions architectures related to the Transformer, information that is not unique to [VSP+17]. The fact that passages can be representative of the paper they are drawn from to varying degrees (e.g. where $\mathcal{P}_2$ would be a better represenatation of [VSP+17] than $\mathcal{P}_1$) makes it necessary for the passage retriever model to be able to distinguish between such better and worse representations. ScholarCopilot does not have to be able to make this distinction since it makes use of the fact that abstracts are inherently representative the paper they are taken from (see [CO06]). Because of this, the PR module would likely benefit from a more sophisticated architecture that is tailored to distinguish passages based on their representativeness. This could potentially be accomplished by using a specifically trained reranker together with the passage retriever or by directly training the retriever itself (which comes with its own challenges, see below). It has to be taken into account however, that passages that are good representations of their paper may not always be the most relevant reference for a given context. Consider the context C from [CLL+23] which cites [DBK+20]:

```
Motivated by successes of transformer (Vaswani et al. 2017) in NLP,
  researchers develop vision transformer (ViT) (Dosovitskiy et al. 2020)
  for image recognition. However, the lack of inductive bias <|CIT-MASK|>
```

A passage like

```
We note that Vision Transformer has much less image-specific inductive
  bias than CNNs. In CNNs, locality, two-dimensional neighborhood
  structure, and translation equivariance are baked into each layer
  throughout the whole model. In ViT, only MLP layers are local and
  translationally equivariant, while the self-attention layers are global.
```

[DBK+20, 3.1] can reasonably be considered more relevant to C than a passage describing the Vision Transformer [DBK+20] in general, like

```
Inspired by the Transformer scaling successes in NLP, we experiment with
  applying a standard Transformer directly to images, with the fewest
  possible modifications. To do so, we split an image into patches and
  provide the sequence of linear embeddings of these patches as an input
  to a Transformer.
```

[DBK+20, 1]. Having to decide whether to retrieve a representative reference or one that provides details about a cited topic makes the task of idenifying relevant passages very challenging, especially in comparison to retrieving relevant abstracts (which are assumed to always be representative).

Runtime performance. As shown in the evaluation, the PR module tends to perform better when given fewer passages. This alone motivates future work on reducing the number of candidate passages compared to simply subdividing a document evenly.

In addition, the PR module performs – by using Batched Self-Constistency [KDSR25] – a lot of redundant computations during citation retrieval. Precomputing all passage representations and retrieving them using an HNSW index like ScholarCopilot would significantly cut down the inference runtime. The evaluation results do not suggest that the PR module can noticably improve generative academic writing with citations, at least not without significant prior modification. Because of this, I did not run dedicated simulations to measure the impact of using the PR module on the effective runtime of ScholarCopilot.However, with the settings used in the experiments, I observed that using the PR module results in citation retrieval taking about ten times as long as using only ScholarCopilot. Evaluating how the PR module affects the runtime of ScholarCopilot is left for future work. Another simple way of reducing the overall runtime would be applying a context window to the context used for retrieval by ScholarCopilot, similar to how it is done for the PR module. This would not only reduce the runtime of ScholarCopilot, but may also potentially be beneficial for the models' citation recall (since the most relevant information to a citation is usually in its immediate proximity, see for example section A.4).

LLM-based generation evaluation. As shown in the evaluation, the decoder model `DeepSeek-R1-Distill-Qwen-7B` can be used for automated generation evaluation. Querying the model to generate textual reasoning for its evaluation does enhance its scoring performance as it consistently reduces the variance between the computed scores when the decoder is tasked to evaluate one generated paper multiple times. However, randomly sampling generated reasoning (see section A.6) reveals that the evaluated models show *degeneration*, that is, they generate incoherent text or get stuck generating repetitive sequences [JLF+23] but that this behaviour is not captured and reflected in the scoreing computed by the judge. This shows that automated generation evaluation with the model used in this thesis does occationally not agree well with human judgement and is thus insufficient as the sole evaluation metric for generation quality. This thesis does by no means provide a sufficient analysis of how well the judge reasons or whether the text generated by ScholarCopilot with or without PR does satisfy any qualitative criteria commonly imposed on academic writing. Since LLM-based generation quality evaluation has been found to be insufficient, future work should provide human expert quality evaluation and in addition could also investigate the performance of larger models for generation evaluation. Contrastive generation evaluation by querying the judge in the same prompt to compare both generated texts against each other (the here conducted evaluation only tasks the judge to compare a candidate generation to the human-written ground truth, see section A.3) may also provide valuable insight.

Training. The ScholarCopilot model is finetuned on a dataset of 500k computer science papers [WMN+25] (59% of which are also present in the HDT dataset [HFB+24] that the samples for all evaluation datasets are drawn, see section 3.2). The model is applied to solve the same tasks as whose that where used to evaluate it in [WMN+25], namely citation retrieval and open-end generation. Thus it is already optimized for these tasks. In contrast, the model used for passage retrieval in the PR

module is an instruction-tuned `Qwen2.5-7B-Instruct` model that is not specifically fine-tuned for the task of distinguishing fitting passages for a given left-hand context (see the examples above, as well as section 4.2). Attributing (generated) statements to supporting evidence is a difficult task for LLMs [LZL23] and to the best of my knowledge, there exists no training or evaluation dataset that maps scientific citing context to the corresponding relevant subsections of the cited documents. Further research is needed to construct such a dataset and in doing so enable the PR module to eventually provide a meaningful improvement over citation retrieval based on abstracts. ContextCite [CWSGM24], which was originally proposed to map statements generated by a RAG model to the source context that caused their generation, could potentially be utilized to build this dataset without having to rely solely on time- and cost-intensive expert annotation. Another possible improvement of the ScholarCopilot-to-PR-module pipeline would be to train ScholarCopilot to directly retrieve and incorporate passages instead of abstracts. This would make the inference much faster since now only has one forward pass through a decoder per retrieval instead of two, but also requires a high-quality training dataset for passage retrieval.

Usability. Finally, one feature I originally planned to implement was to query the passage retriever during citation retrieval to provide reasoning why it chooses to cite a specific document. This would make it much easier for a human reader to validate whether a generated citation ... TODO

6. Conclusion

TODO

A. Appendix

A.1. Hyperparameters

TODO

A.2. Passage Retrieval Instruction

This instruction is based on the `bright_general`[1] instruction proposed for ReasonIR [SQK+25], which is included as "Prompt for reranking StackExchange" in the paper. Fields marked with {} are filled during retrieval.

```
You are given a query and {} paragraphs. Assign each paragraph a score
  based how likely it is for the given paragraph to be cited at the end
  of the query. A paragraph should receive a high score if it explains
  the same or a topic similar to what is discussed towards the end of the
  query. Following the order of the passages below, your answer should be
  '[..., xi, ...]' where each xi is a number from 0-10 and the score of
  the corresponding paragraph-i. 0 means completely irrelevant, 10 means
  highly relevant and very likely to be cited at the end of the query.
  Don't output anything else. Output exactly {} scores. Here is the query:
<start_query>
{}
<end_query>
Here are the paragraphs:
<start_paragraph-1>
{}
<end_paragraph-1><start_paragraph-2>
{}
<end_paragraph-2>
[...]
```

A.3. Generation Evaluation Instruction

These instructions are based on the "Generation Quality Evaluation prompt" proposed together with ScholarCopilot [WMN+25]. Note that every time the evaluation

[1] `https://github.com/facebookresearch/ReasonIR/blob/main/evaluation/bright/prompts.py`

dimensions are listed, this list is shuffled and ordered randomly before the prompt is passed to the judge. This is part of the implementation of Batched Self-Consistency [KDSR25] for the judge. Fields marked with {} are filled during evaluation.

Instruction querying for reasoning

```
You are a senior computer science scholar. Please evaluate the
  AI-generated content using the ground truth as reference. Evaluate the
  following five dimensions by comparing the AI-generated content with
  the ground truth:
[Detailed Evaluation]
1. Content Relevance:
  - Key strengths:
  - Main gaps:
  - Comparison with ground truth:
2. Logical Coherence:
  - Key strengths:
  - Main gaps:
  - Comparison with ground truth:
3. Academic Rigor:
  - Key strengths:
  - Main gaps:
  - Comparison with ground truth:
4. Background Completeness:
  - Key strengths:
  - Main gaps:
  - Comparison with ground truth:
5. Innovation Statement:
  - Key strengths:
  - Main gaps:
  - Comparison with ground truth:
[End Evaluation]
[Improvement Suggestions]
  1.
  2.
  3.
[End Suggestions]
Based on your above analysis, provide numerical scores in the following
  format:
[Scores]
Content Relevance: <score>/5
Logical Coherence: <score>/5
Academic Rigor: <score>/5
Background Completeness: <score>/5
Innovation Statement: <score>/5
Total: <sum>/25
[End Scores]
Below are the materials for evaluation:
Paper Title:
{}
```

```
Abstract:
{}
Ground Truth Content:
{}
AI Generated Content:
{}
Remember to first provide detailed evaluation, then improvement
  suggestions, and finally the numerical scores in the exact format
  specified above.
Make sure that you output the numerical scores last.
```

Instruction querying only for scores

```
You are a senior computer science scholar. Please evaluate the
  AI-generated content using the ground truth as reference. Evaluate the
  following five dimensions by comparing the AI-generated content with
  the ground truth.
Based on your analysis, provide numerical scores in the following format:
[Scores]
Content Relevance: <score>/5
Logical Coherence: <score>/5
Academic Rigor: <score>/5
Background Completeness: <score>/5
Innovation Statement: <score>/5
Total: <sum>/25
[End Scores]
Below are the materials for evaluation:
Paper Title:
{}
Abstract:
{}
Ground Truth Content:
{}
AI Generated Content:
{}
Remember to provide the numerical scores in the exact format specified
  above.
Make sure that you output the numerical scores last.
```

A.4. Complete Example of Single-Step Retrieval

Shown below are the full context and retrieved references for the example that was used in section 4.2:

Retrieval context

```
1 Introduction Task-oriented dialog (TOD) systems have been widely
  deployed for popular virtual assistants, like Siri and Google
  Assistant. They help people with tasks such as booking restaurants and
  looking for a hotel by searching databases with constraints provided by
  users. After retrieving a result from the database, a system may
  continue by conducting actions like making a reservation or providing
  more information about receiving the result. However, there can be
  multiple results from the database that match the same constraints. For
  example, as shown in Fig.1, the system finds two available hotels at
  different locations when the user is asking the system to help book a
  hotel. This kind of ambiguity stops system from proceeding until the
  system finds out which result the user looks for. Therefore, we need to
  enhance the system with the ability to resolve such ambiguity brought
  out by multiple items returned from database search. We call this type
  of ambiguity as database search result ambiguity (DSR-ambiguity).
  Different from semantic ambiguous words (e.g. orange can be referred as
  either color or fruit), the DSR-ambiguity focuses on results from
  multiple database search results. Solving such disambiguation tasks
  consists of two steps: asking clarification questions and understanding
  users corresponding answers. While there is a relatively larger body of
  literature focusing on when and how to give out the clarification
  question #ref23# #ref22# #ref13#, the focus on understanding users
  answers/intents has been relatively sparse. Our work mainly focuses on
  improving models ability of understanding the answers by augmenting two
  existing task-oriented dialog datasets: MultiWOZ <|CIT-MASK|> and
  Schema-Guided Dataset (SGD) #ref25#. MultiWOZ and SGD are the most
  popular large-scale task-oriented dialog datasets, based on which most
  of the state-of-the-art dialog system models are commonly trained and
  evaluated. According to our analysis, there are around 66[percent]
  dialogs of the dataset contains multiple dataset-searching results,
  which means the DSR-ambiguity exists. In this setting, ambiguities are
  skipped and the model trained based on these datasets can hardly handle
  the cases where users prefer to make their own choices among all the
  results satisfies the constraints. Furthermore, users should be given
  more detailed information about search results. Ideally, dialog models
  should provide the information and assist users to make choices, rather
  than picking one from the result list and recommending it to users. It
  is not necessary to list all the results, but enumerating 2 or 3
  options would help increase users engagement. To strengthen the model
  with the ability to handle the ambiguity, we propose to augment these
  two datasets with disambiguation turns, where the system provides all
  possible matched results and lets the user make their own decision
  based on the complete information. Specifically, we first extract
  templates from the SIMMC 2.0 dataset <|CIT-MASK|>
```

Reference retrieved by SC

```
Abstract This paper presents our work on the Situated Interactive
  MultiModal Conversations 2.0 challenge held at Dialog State Tracking
  Challenge 10. SIMMC 2.0 includes 4 subtasks, and we introduce our
```

multimodal approaches for the subtask #1, #2 and the generation of subtask #4. SIMMC 2.0 dataset is a multimodal dataset containing image and text information, which is more challenging than the problem of only text-based conversations because it must be solved by understanding the relationship between image and text. Therefore, since there is a limit to solving only text models such as BERT or GPT2, we propose a multimodal model combining image and text. We first pretrain the multimodal model to understand the relationship between image and text, then finetune our model for each task. We achieve the 3rd best performance in subtask #1, #2 and a runner-up in the generation of subtask #4. The source code is available at https://github.com/rungjoo/simmc2.0.

Reference retrieved by SC+PR

The aim of the SIMMC 2.0 dataset is to emulate futuristic, real-world shopping scenarios where humans converse with a dialog agent in natural language grounded in a situated multimodal context. As a step towards this intelligent conversational agent, we leverage the dialogs and annotations in our dataset and propose four benchmark tasks (summarized in Tab.4) along with evaluation metrics. These tasks capture several multimodal conversational reasoning challenges, as elaborated next. In a real-world conversation, humans often use shorthands (coreferences) in order to refer to objects / events that have already been mentioned in the dialog. While we reserve modeling coreference resolution as a challenging task in Sec.4.2, it is important for the system to recognize ambiguous uses of such coreferences even before attempting to resolve them. For example, consider A: The blue trousers are priced at $45. U: What about those?, where the phrase those could be ambiguous in the following situations: (a) The user refers to a group of trousers without specifying the exact one they have in mind, (b) The user incorrectly uses a shorthand for a novel pair of trousers not mentioned in the dialog due to conversational brevity. In either cases, identifying the need for disambiguation and responding with Which ones are you talking about? The red or the green pair? is a desirable trait for a robust assistant system. The multimodal disambiguation task tests this ability of the agent. More concretely, given the dialog history and the current user utterance, multimodal disambiguation requires the agent to predict a binary label conditioned on the multimodal context, to indicate the presence of a referential ambiguity in the user utterance. This label could also be useful for other downstream tasks like assistant response generation (Sec.4.4) in order to continue the conversation in a meaningful way. We use accuracy to measure and compare model performances for this task. For this task, we aim to resolve referential mentions in user utterances to their canonical object IDs as defined for each scene. These mentions can be resolved through (1) the dialog context (e.g. A: This shirt comes in XL and is $29. #eq50# U: Please add it to cart., or (2) the multimodal context (e.g. U: How much is that red shirt?), or (3) both (e.g. U: How much is the one next to the one you mentioned?). The input for

Note that "%" is replaced with `[percent]` due to formatting issues.

A.5. Complete Example of LLM-based Generation Quality Evaluation with Reasoning

TODO

Original Introduction of the paper

```
Several applications involve analyzing many interacting entities, each
  generating a large quantity of multi-variate time-series data. Learning
  the relationships among these entities, especially in real-time, is
  essential for pattern discovery, prediction, and correlation-based
  clustering. In distributed scenarios, each agent (observer) can only
  collect data from a subset of the entities and communicate with its
  neighbors within a specific communication range to learn the
  relationships of the entire network in real-time (i.e., online). Such
  distributed online correlation inference has many applications, for
  example, in covariance-based and distributed clustering #ref1# #ref2#,
  traffic network prediction and tracking #ref3# #ref4#, social network
  inference #ref5#, and biomedical sensor data inference #ref6#. To learn
  the relationships between entities, one approach is to represent each
  entity as a node in a graph, and each edge defines an interaction
  between the nodes. Learning the network topology from data is studied
  extensively in the literature #ref7# #ref8# #ref9# #ref10#. One method
  focuses on estimating the non-zero elements of the sparse inverse
  covariance matrix of these random variables (i.e., nodes) #ref7#
  #ref11# through inferring the edges of the graph by using an
  #eq0#-regularized Gaussian maximum likelihood estimator, assuming the
  random variables are jointly Gaussian. In this case, an edge between
  nodes indicates that the corresponding random variables are
  conditionally dependent, given all the other variables. This is known
  as the graphical lasso problem #ref12# #ref13# #ref14#. Various
  optimization algorithms have been proposed to solve this problem,
  including coordinate descent #ref13#, proximal methods #ref15# #ref16#,
  and alternating minimization methods #ref17# #ref18#. However, these
  algorithms have not been studied extensively for distributed and online
  inference. In #ref19#, an online sparse inverse covariance algorithm
  was proposed, but it is not directly applicable to the distributed
  setting. In this paper, given multi-variate time-series data gathered
  by each agent in a network, we propose a peer-to-peer distributed
  algorithm for each agent to estimate the underlying relationships
  between all the network variables in real-time via distributed and
  online inverse covariance matrix estimation. We provide theoretical
  guarantees on the convergence of the estimates and characterize the
```

```
  asymptotic rate of convergence. We also demonstrate the performance of
  our proposed algorithm through simulations.
```

Text generated by SC

```
Several applications involve analyzing many interacting entities, each
  generating a large quantity of multi-variate time-series data. Learning
  the relationships among these entities, especially in real-time, is
  essential for pattern discovery, prediction, and correlation-based
  clustering. In distributed scenarios, each agent (observer) can only
  collect data from a subset of the entities and communicate with its
  neighbors. In this paper, we address this challenge by developing a
  distributed algorithm that learns the network structure (i.e.,
  dependencies among observed entities) in real-time from data collected
  by distributed agents.\n\n\\subsection{Background and Related
  Work}\nGraphical models are a powerful tool to represent the
  relationship between different entities in a network. In particular,
  Gaussian graphical models (GGMs) represent the conditional dependencies
  between entities in a multivariate Gaussian distribution by a graph,
  where the absence of an edge between two nodes implies conditional
  independence between the corresponding entities given all other
  entities \\cite{arxivID-1606.07282-0}. The inverse of the covariance
  matrix (called the precision matrix) of the multivariate Gaussian
  distribution represents the conditional dependencies in GGMs. The
  sparsity pattern of the precision matrix can be exploited to infer the
  conditional dependencies among entities. However, inferring the
  structure of the precision matrix is a challenging task since it is an
  NP-hard problem \\cite{arxivID-1303.1501-1}.\n\nMany algorithms have
  been developed to infer the structure of the precision matrix. Some of
  them rely on the assumption that the data follows a GGM and the
  precision matrix is sparse. These algorithms are based on the
  assumption that the precision matrix is sparse and try to estimate the
  structure by solving a convex optimization problem. A popular algorithm
  in this category is graphical lasso \\cite{arxivID-0707.0704-2}, which
  solves a convex optimization problem to estimate the precision matrix.
  Another class of algorithms that can be used to estimate the structure
  of the precision matrix are graphical alternating minimization (GAM)
  algorithms \\cite{arxivID-1212.2462-3, arxivID-1207.4118-4,
  arxivID-1302.1524-5, arxivID-1807.00944-6, arxivID-1608.02315-7,
  arxivID-1301.3720-8, arxivID-1209.6561-9, arxivID-1206.6431-10,
  arxivID-2006.13843-11, arxivID-2202.09691-12, arxivID-1206.5247-13,
  arxivID-1501.04370-14, arxivID-1205.2612-15, arxivID-2110.04719-16,
  arxivID-2110.04719-17, arxivID-2103.12188-18, arxivID-1309.6820-19,
  arxivID-1505.02870-20, arxivID-1206.6452-21, arxivID-2108.08871-22,
  arxivID-2006.05691-23}
```

Text generated by SC+PR

```
Several applications involve analyzing many interacting entities, each
  generating a large quantity of multi-variate time-series data. Learning
```

```
    the relationships among these entities, especially in real-time, is
    essential for pattern discovery, prediction, and correlation-based
    clustering. In distributed scenarios, each agent (observer) can only
    collect data from a subset of the entities and communicate with other
    agents. This paper develops a distributed algorithm to learn the
    relationships between these entities in real-time from the data
    collected by the agents.\n\nThe relationships among entities can be
    modeled as a Gaussian graphical model, where each node represents an
    entity and the edges represent the dependencies between the entities.
    The inverse covariance matrix of the multivariate Gaussian distribution
    is the essential parameter in the model, and its sparsity pattern is
    the desired structure in the model. The structure is a sparse graph and
    can be used to identify the relationships among the entities. Existing
    algorithms to learn the structure of Gaussian graphical models are
    based on the $l_1$-regularized maximum likelihood estimation
    \\cite{arxivID-1209.6419-0, arxivID-1209.6419-1, arxivID-1209.6419-2,
    arxivID-1209.6419-3, arxivID-1209.6419-4}. These algorithms are
    centralized and batch-based, requiring all data to be gathered at a
    central location before processing.\n\nThe focus of this paper is to
    develop a distributed online algorithm to learn the sparse inverse
    covariance matrix of a multivariate Gaussian distribution. Our
    algorithm is built on the graphical alternating minimization (GAM)
    algorithm \\cite{arxivID-2109.12020-5}, which is an online algorithm to
    learn the sparse inverse covariance matrix. In GAM, the algorithm
    parameters are updated over time based on the current data point and
    the previous estimates. GAM alternates between estimating the precision
    matrix and updating the mean of the data. The algorithm is online,
    meaning that it updates the estimates as new data points arrive. GAM is
    based on the alternating direction method of multipliers (ADMM)
    \\cite{arxivID-1308.3558-6}, an optimization algorithm that solves
    large-scale optimization problems by breaking them into smaller
    subproblems that can be solved in parallel. ADMM is a powerful
    optimization technique that has been used in a wide range of
    applications, including image processing, signal processing, and
    machine learning.
  The algorithm consists of two steps: the dual ascent step and the
    proximal step. In the dual ascent step, the algorithm updates the dual
    variables, which are used to enforce the constraints of the
    optimization problem. In the proximal step, the algorithm updates the
    primal variables, which are the variables of interest. The algorithm
    iterates between these two steps until convergence is achieved. ADMM is
    particularly useful for solving optimization problems that involve
    large-scale data, as it can be parallelized and distributed across
    multiple processors. This makes it a popular choice for big data
    applications. ADMM has been used in a variety of applications,
    including image processing, signal processing, and machine learning. In
    image processing, ADMM has been used for image restoration, denoising,
    and super-resolution. In signal processing, ADMM has been used for
    signal denoising, deconvolution, and compressed sensing. In machine
```

```
learning, ADMM has been used for regularized regression, support vector
machines, and clustering. Overall, ADMM is a powerful optimization
technique that has been used in a wide range of applications. Its
ability to handle large-scale data and its parallelizability make it a
popular choice for big data applications.\n\nThe paper is organized as
follows. Section \\ref{sec:formulation} introduces the problem
formulation. Section \\ref{sec:algorithm} presents the distributed
algorithm. Section \\ref{sec:convergence} analyzes the convergence of
the algorithm. Section \\ref{sec:simulation} shows simulation results.
Section \\ref{sec:conclusion} concludes the paper. <|section_end|>
```

Generated evaluation of the text generated by SC

```
<think>\nOkay, I'm trying to evaluate this AI-generated content based on
the ground truth. The paper is titled \"Distributed Estimation of
Sparse Inverse Covariances.\" Let me go through each dimension one by
one.\n\nFirst, the Innovation Statement. The AI content mentions
developing a distributed algorithm for real-time inverse covariance
estimation but doesn't compare it with the ground truth. The ground
truth paper proposes a peer-to-peer distributed algorithm with
theoretical guarantees. The AI lacks specific gaps or strengths
compared to the ground truth, so I'll rate it 2/5.\n\nNext, Academic
Rigor. The AI mentions developing an algorithm and provides some
references, but it's quite brief. The ground truth paper is more
comprehensive, discussing various algorithms and theoretical aspects.
The AI could improve by expanding on these points. I'll give it
3/5.\n\nFor Logical Coherence, the AI's content is somewhat coherent
but lacks depth and connections to the ground truth. It jumps between
concepts without clear flow. The ground truth paper is logically
structured, so the AI needs more coherence. I'll score this
2/5.\n\nBackground Completeness. The AI introduces graphical models and
mentions the graphical lasso but doesn't connect them to the ground
truth. The ground truth provides a thorough background. The AI could
improve by better linking concepts. I'll rate it 2/5.\n\nRegarding
Content Relevance, the AI's content is somewhat relevant but lacks
specific details about the distributed aspect. It doesn't mention
consensus terms or communication rounds as the ground truth does. I'll
give it 3/5.\n\nIn improvement suggestions, the AI should connect its
approach to the ground truth, expand on academic rigor by discussing
limitations, improve logical flow, better explain background, and
include specific algorithmic details.\n\nFinally, assigning scores:
Innovation Statement 2/5, Academic Rigor 3/5, Logical Coherence 2/5,
Background Completeness 2/5, Content Relevance 3/5. Totaling to
12/25.\n</think>\n\n[Scores]\nInnovation Statement: 2/5\nAcademic
Rigor: 3/5\nLogical Coherence: 2/5\nBackground Completeness:
2/5\nContent Relevance: 3/5\nTotal: 12/25\n[End
Scores]<|end_of_sentence|>
```

Generated evaluation of the text generated by SC+PR

```
<think>\nOkay, I'm going to evaluate the AI-generated content against
  the ground truth. Let's start with Content Relevance. The AI paper
  seems to cover a lot of ground, introducing distributed estimation for
  sparse inverse covariances, which is relevant. However, it's a bit too
  detailed and could be more concise. It misses some of the key points
  from the ground truth, like the applications and the specific
  algorithms used in distributed settings. So, I'd give it a
  3/5.\n\nNext, Background Completeness. The AI content does mention
  several algorithms, but it doesn't tie them back to the distributed
  setting as clearly as the ground truth. It's missing some of the
  references and the specific context of distributed online inverse
  covariance estimation. I'd rate this a 2/5.\n\nFor Innovation
  Statement, the AI paper proposes a distributed online algorithm using
  GAM and ADMM, which is innovative. However, it doesn't highlight the
  unique aspects of their approach compared to existing methods. It's
  somewhat vague on how their method improves upon previous work. So, a
  2/5 here.\n\nAcademic Rigor is decent. The AI uses technical terms and
  mentions optimization techniques, which adds depth. But it lacks
  detailed mathematical formulations and proofs, which would make it less
  rigorous. I'd give this a 3/5.\n\nLogical Coherence is good. The
  structure is clear, moving from introduction to problem, method,
  algorithm, convergence, and simulations. It flows logically, but the
  explanations could be more precise. I'd rate this a 4/5.\n\nMoving on
  to Improvement Suggestions. The AI could better integrate references,
  especially those from the ground truth. It should provide more
  mathematical details and compare its method with existing literature
  more thoroughly. Also, it should clarify how the distributed approach
  specifically benefits the problem.\n\nNow, for the numerical scores.
  Summing up the individual scores: Content Relevance 3, Background
  Completeness 2, Innovation Statement 2, Academic Rigor 3, Logical
  Coherence 4. Total is 14/25.\n</think>\n\n[Scores]\nContent Relevance:
  3/5\nBackground Completeness: 2/5\nInnovation Statement: 2/5\nAcademic
  Rigor: 3/5\nLogical Coherence: 4/5\nTotal: 14/25\n\n[Improvement
  Suggestions]\n1. Integrate more references from the ground truth.\n2.
  Provide more mathematical details and proofs.\n3. Compare the method
  with existing literature more thoroughly.\n4. Clarify the specific
  benefits of the distributed approach.\n\n[End
  Evaluation]<|end_of_sentence|>
```

A.6. "False Positive" Example of LLM-based Generation Quality Evaluation

TODO

The expected behaviour would be that the judge assigns a lower `Logical Coherence` score to the text generated by SC+PR (which is not the case since $4 > 2.8$) and that

this score is on the lower end of the permitted scale (which is also false since $4 > 2.5$ when we define 'lower end' of the interval $[0,5]$ to be $[0,2.5)$). Note that generating long sequences of consecutive citations or getting stuck in generating repetitive sequences are not issues unique to ScholarCopilot and the PR module respectively, both models occationally show such behaviour.

Part of the Introduction generated by SC

```
[...]
In this paper, we propose a reversible adversarial example generator
  (RAEG) that introduces slight changes to the images to fool traditional
  classification models. Even though malicious attacks train pirated
  models based on the defensed versions of the protected images, RAEG can
  significantly weaken the functionality of these models. Meanwhile, the
  reversibility of RAEG ensures the performance of authorized models.
  Therefore, the protected data can be used for training the authorized
  models, but not for pirated models.\n\nThe second problem is how to
  defend against adversarial defense methods. Adversarial defense methods
  aim to eliminate the adversarial perturbations from adversarial
  examples to recover the original data. These methods can be divided
  into two categories: adversarial training \\cite{arxivID-1705.03387-0,
  arxivID-2310.05947-1, arxivID-1905.11713-2, arxivID-2109.09901-3,
  arxivID-1906.11567-4, arxivID-1711.08001-5, arxivID-2007.14672-6,
  arxivID-2003.02484-7}and adversarial purification
  \\cite{arxivID-1912.05391-8, arxivID-1612.07767-9,
  arxivID-1804.01635-10, arxivID-2405.15971-11, arxivID-2405.15971-12,
  arxivID-2212.05630-13, arxivID-2406.03143-14}.
[...]
```

Part of the Introduction generated by SC+PR

```
[...]
We develop a reversible adversarial example generator (RAEG) that
  introduces slight changes to the images to fool traditional
  classification models. The changes are imperceptible to the human eyes.
  The protected images cannot be used to train models directly. However,
  the protected images can be reversely transformed into the original
  images. The original images can be used to train models and the
  authorized models can achieve good performance. Meanwhile, the
  protected images cannot be used to train pirated models. The pirated
  models trained based on the protected images cannot achieve
  satisfactory performance. The pipeline of our proposed method is shown
  in Figure~\\ref{fig1}. The protected images can be used to train models
  and the authorized models can achieve good performance. Meanwhile, the
  protected images cannot be used to train pirated models. The pirated
  models trained based on the protected images cannot achieve
  satisfactory performance.
[...]
\\textbf{Adversarial Training:} Adversarial training is a proactive
  defense method. Adversarial training is used to train models that are
```

```
  robust to adversarial attacks. Adversarial training is a widely used
  defense method. Adversarial training is used to train models that are
  robust to adversarial attacks. Adversarial training is a widely used
  defense method. Adversarial training is used to train models that are
  robust to adversarial attacks. Adversarial training is a widely used
  defense method.
[...]
```

Scores computed by the judge


```
"sc_scores": {
  "Content Relevance": 3.4,
  "Logical Coherence": 2.8,
  "Academic Rigor": 3.0,
  "Background Completeness": 2.8,
  "Innovation Statement": 2.4,
  "total": 13.4
},
"sc_pr_scores": {
  "Content Relevance": 4.2,
  "Logical Coherence": 4.0,
  "Academic Rigor": 3.4,
  "Background Completeness": 3.6,
  "Innovation Statement": 4.4,
  "total": 18.6
}
```

Bibliography

[AHS+24] Akari Asai, Jacqueline He, Rulin Shao, Weijia Shi, Amanpreet Singh, Joseph Chee Chang, Kyle Lo, Luca Soldaini, Sergey Feldman, Mike D'arcy, David Wadden, Matt Latzke, Minyang Tian, Pan Ji, Shengyan Liu, Hao Tong, Bohao Wu, Yanyu Xiong, Luke Zettlemoyer, Graham Neubig, Dan Weld, Doug Downey, Wen tau Yih, Pang Wei Koh, and Hannaneh Hajishirzi. Openscholar: Synthesizing scientific literature with retrieval-augmented lms. *arXiv preprint arXiv:2411.14199*, 2024.

[CLL+23] Mengzhao Chen, Mingbao Lin, Ke Li, Yunhang Shen, Yongjian Wu, Fei Chao, and Rongrong Ji. Cf-vit: A general coarse-to-fine method for vision transformer. In *Proceedings of the AAAI conference on artificial intelligence*, volume 37, pages 7042–7052, 2023.

[CO06] Cate Cross and Charles Oppenheim. A genre analysis of scientific abstracts. *Journal of Documentation*, 62:428–446, 07 2006.

[CWSGM24] Benjamin Cohen-Wang, Harshay Shah, Kristian Georgiev, and Aleksander Madry. Contextcite: Attributing model generation to context. *Advances in Neural Information Processing Systems*, 37:95764–95807, 2024.

[DAGY+25] DeepSeek-AI, Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, Xiaokang Zhang, Xingkai Yu, Yu Wu, Z. F. Wu, Zhibin Gou, Zhihong Shao, Zhuoshu Li, Ziyi Gao, Aixin Liu, Bing Xue, Bingxuan Wang, Bochao Wu, Bei Feng, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, Damai Dai, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Han Bao, Hanwei Xu, Haocheng Wang, Honghui Ding, Huajian Xin, Huazuo Gao, Hui Qu, Hui Li, Jianzhong Guo, Jiashi Li, Jiawei Wang, Jingchang Chen, Jingyang Yuan, Junjie Qiu, Junlong Li, J. L. Cai, Jiaqi Ni, Jian Liang, Jin Chen, Kai Dong, Kai Hu, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Liang Zhao, Litong Wang, Liyue Zhang, Lei Xu, Leyi Xia, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Meng Li, Miaojun Wang, Mingming Li, Ning Tian, Panpan Huang, Peng Zhang, Qiancheng Wang, Qinyu Chen, Qiushi Du, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, R. J. Chen, R. L. Jin, Ruyi Chen, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shengfeng Ye, Shiyu Wang, Shuiping Yu,

Shunfeng Zhou, Shuting Pan, S. S. Li, Shuang Zhou, Shaoqing Wu, Shengfeng Ye, Tao Yun, Tian Pei, Tianyu Sun, T. Wang, Wangding Zeng, Wanjia Zhao, Wen Liu, Wenfeng Liang, Wenjun Gao, Wenqin Yu, Wentao Zhang, W. L. Xiao, Wei An, Xiaodong Liu, Xiaohan Wang, Xiaokang Chen, Xiaotao Nie, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xinyu Yang, Xinyuan Li, Xuecheng Su, Xuheng Lin, X. Q. Li, Xiangyue Jin, Xiaojin Shen, Xiaosha Chen, Xiaowen Sun, Xiaoxiang Wang, Xinnan Song, Xinyi Zhou, Xianzu Wang, Xinxia Shan, Y. K. Li, Y. Q. Wang, Y. X. Wei, Yang Zhang, Yanhong Xu, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Wang, Yi Yu, Yichao Zhang, Yifan Shi, Yiliang Xiong, Ying He, Yishi Piao, Yisong Wang, Yixuan Tan, Yiyang Ma, Yiyuan Liu, Yongqiang Guo, Yuan Ou, Yuduan Wang, Yue Gong, Yuheng Zou, Yujia He, Yunfan Xiong, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Y. X. Zhu, Yanhong Xu, Yanping Huang, Yaohui Li, Yi Zheng, Yuchen Zhu, Yunxian Ma, Ying Tang, Yukun Zha, Yuting Yan, Z. Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhicheng Ma, Zhigang Yan, Zhiyu Wu, Zihui Gu, Zijia Zhu, Zijun Liu, Zilin Li, Ziwei Xie, Ziyang Song, Zizheng Pan, Zhen Huang, Zhipeng Xu, Zhongyu Zhang, and Zhen Zhang. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.

[DBK+20] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929*, 2020.

[DCLT19] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*, pages 4171–4186, 2019.

[DGD+24] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. The faiss library. *arXiv preprint arXiv:2401.08281*, 2024.

[DOW+24] Hyo Jin Do, Rachel Ostrand, Justin D Weisz, Casey Dugan, Prasanna Sattigeri, Dennis Wei, Keerthiram Murugesan, and Werner Geyer. Facilitating human-llm collaboration through factuality scores and source attributions. *arXiv preprint arXiv:2405.20434*, 2024.

[HFB+24] Haoyu He, Markus Flicke, Jan Buchman, Iryna Gurevych, and Andreas

Geiger. HDT: Hierarchical Document Transformer. Conference on Language Modeling, 2024.

[HPS+25] Sebastian Heineking, Jonas Probst, Daniel Steinbach, Martin Potthast, and Harrisen Scells. Ranking generated answers: On the agreement of retrieval models with humans on consumer health questions. In *European Conference on Information Retrieval*, pages 128–137. Springer, 2025.

[HRBB23] Jakob Drachmann Havtorn, Amélie Royer, Tijmen Blankevoort, and Babak Ehteshami Bejnordi. Msvit: Dynamic mixed-scale tokenization for vision transformers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 838–848, 2023.

[JLF+23] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Ye Jin Bang, Andrea Madotto, and Pascale Fung. Survey of hallucination in natural language generation. *ACM computing surveys*, 55(12):1–38, 2023.

[KDSR25] Anton Korikov, Pan Du, Scott Sanner, and Navid Rekabsaz. Batched self-consistency improves llm relevance assessment and ranking. *arXiv preprint arXiv:2505.12570*, 2025.

[KMGD21] Satwik Kottur, Seungwhan Moon, Alborz Geramifard, and Babak Damavandi. Simmc 2.0: A task-oriented dialog dataset for immersive multimodal conversations. *arXiv preprint arXiv:2104.08667*, 2021.

[KOM+20] Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick SH Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *EMNLP (1)*, pages 6769–6781, 2020.

[KP23] Po-Nien Kung and Nanyun Peng. Do models really learn to follow instructions? an empirical study of instruction tuning. *arXiv preprint arXiv:2305.11383*, 2023.

[LH21] Joosung Lee and Kijong Han. Multimodal interactions using pretrained unimodal models for simmc 2.0. *arXiv preprint arXiv:2112.05328*, 2021.

[LLG+19] Mike Lewis, Yinhan Liu, Naman Goyal, Marjan Ghazvininejad, Abdelrahman Mohamed, Omer Levy, Ves Stoyanov, and Luke Zettlemoyer. Bart: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension. *arXiv preprint arXiv:1910.13461*, 2019.

[LOS+23] Jakub Lála, Odhran O'Donoghue, Aleksandar Shtedritski, Sam Cox, Samuel G Rodriques, and Andrew D White. Paperqa: Retrieval-augmented generative agent for scientific research. *arXiv preprint arXiv:2312.07559*, 2023.

[LPP+20] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems*, 33:9459–9474, 2020.

[LZL23] Nelson F Liu, Tianyi Zhang, and Percy Liang. Evaluating verifiability in generative search engines. *arXiv preprint arXiv:2304.09848*, 2023.

[Man78] Jean M. Mandler. A code in the node: The use of a story schema in retrieval. *Discourse Processes*, 1(1):14–35, 1978.

[MHW+24] Niklas Muennighoff, SU Hongjin, Liang Wang, Nan Yang, Furu Wei, Tao Yu, Amanpreet Singh, and Douwe Kiela. Generative representational instruction tuning. In *The Thirteenth International Conference on Learning Representations*, 2024.

[MY18] Yu A Malkov and Dmitry A Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE transactions on pattern analysis and machine intelligence*, 42(4):824–836, 2018.

[QBK+21] Kun Qian, Ahmad Beirami, Satwik Kottur, Shahin Shayandeh, Paul Crook, Alborz Geramifard, Zhou Yu, and Chinnadhurai Sankar. Database search results disambiguation for task-oriented dialog systems. *arXiv preprint arXiv:2112.08351*, 2021.

[RRS20] Adam Roberts, Colin Raffel, and Noam Shazeer. How much knowledge can you pack into the parameters of a language model? *arXiv preprint arXiv:2002.08910*, 2020.

[Rum75] David E. Rumelhart. Notes on a schema for stories. In Daniel G. Bobrow and Allan Collins, editors, *Representation and Understanding*, pages 211–236. Morgan Kaufmann, San Diego, 1975.

[SCM+23] Freda Shi, Xinyun Chen, Kanishka Misra, Nathan Scales, David Dohan, Ed H Chi, Nathanael Schärli, and Denny Zhou. Large language models can be easily distracted by irrelevant context. In *International Conference on Machine Learning*, pages 31210–31227. PMLR, 2023.

[SHC+24] Aviv Slobodkin, Eran Hirsch, Arie Cattan, Tal Schuster, and Ido Dagan. Attribute first, then generate: Locally-attributable grounded text generation. *arXiv preprint arXiv:2403.17104*, 2024.

[SM90] Françoise Salager-Meyer. Discoursal flaws in medical english abstracts: A genre analysis per research- and text-type. *Text - Interdisciplinary Journal for the Study of Discourse*, 10, 01 1990.

[SQK+25] Rulin Shao, Rui Qiao, Varsha Kishore, Niklas Muennighoff, Xi Victoria

Lin, Daniela Rus, Bryan Kian Hsiang Low, Sewon Min, Wen-tau Yih, Pang Wei Koh, and Luke Zettlemoyer. Reasonir: Training retrievers for reasoning tasks. *arXiv preprint arXiv:2504.20595*, 2025.

[sta25a] arXiv staff. About arXiv, 2025. `https://info.arxiv.org/about/index.html`, Last accessed: 10.09.2025.

[sta25b] arXiv staff. Understanding the arXiv identifier, 2025. `https://info.arxiv.org/help/arxiv_identifier.html`, Last accessed: 15.09.2025.

[STB25] Shubham Sharma, Sneha Tuli, and Narendra Badam. Challenges and applications of large language models: A comparison of gpt and deepseek family of models. *arXiv preprint arXiv:2508.21377*, 2025.

[TLI+23] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. LLaMA: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.

[VSP+17] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

[WMN+25] Yubo Wang, Xueguang Ma, Ping Nie, Huaye Zeng, Zhiheng Lyu, Yuxuan Zhang, Benjamin Schneider, Yi Lu, Xiang Yue, and Wenhu Chen. Scholarcopilot: Training large language models for academic writing with accurate citations. *arXiv preprint arXiv:2504.00824*, 2025.

[XLS+24] Fangyuan Xu, Kyle Lo, Luca Soldaini, Bailey Kuehl, Eunsol Choi, and David Wadden. Kiwi: A dataset of knowledge-intensive writing instructions for answering research questions. *arXiv preprint arXiv:2403.03866*, 2024.

[XWL+24] Sirui Xia, Xintao Wang, Jiaqing Liang, Yifei Zhang, Weikang Zhou, Jiaji Deng, Fei Yu, and Yanghua Xiao. Ground every sentence: Improving retrieval-augmented llms with interleaved reference-claim generation. *arXiv preprint arXiv:2407.01796*, 2024.

[YYZ+24] An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jingren Zhou, Junyang Lin, Kai Dang, Keming Lu, Keqin Bao, Kexin Yang, Le Yu, Mei Li, Mingfeng Xue, Pei Zhang, Qin Zhu, Rui Men, Runji Lin, Tianhao Li, Tianyi Tang, Tingyu Xia, Xingzhang Ren, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yu Wan, Yuqiong

Liu, Zeyu Cui, Zhenru Zhang, and Zihan Qiu. Qwen2 technical report. *arXiv preprint arXiv:2407.10671*, 2024.

[ZBL+24] Jiajie Zhang, Yushi Bai, Xin Lv, Wanjun Gu, Danqing Liu, Minhao Zou, Shulin Cao, Lei Hou, Yuxiao Dong, Ling Feng, and Juanzi Li. Longcite: Enabling llms to generate fine-grained citations in long-context qa. *arXiv preprint arXiv:2409.02897*, 2024.

[ZDL+25] Shengyu Zhang, Linfeng Dong, Xiaoya Li, Sen Zhang, Xiaofei Sun, Shuhe Wang, Jiwei Li, Runyi Hu, Tianwei Zhang, Fei Wu, and Guoyin Wang. Instruction tuning for large language models: A survey. *arXiv preprint arXiv:2308.10792*, 2025.